BugBusters

Specifica Tecnica$_G$

Versione 0.1.2

Stato	In Redazione
Redattori$_G$	Alberto Pignat, Leonardo Salviato
Verificatori$_G$	Alberto Autiero
Distribuzione	BugBusters, Eggon, Prof. Tullio Vardanega, Prof. Riccardo Cardin

Descrizione

Documento dove viene spiegata in dettaglio la struttura architetturale, le tecnologie utilizzate e le API esposte del prodotto software.

Registro delle Modifiche

Versione	Data	Descrizione	Redatto	Verificato	Approvato
0.1.3	01/04/2026	Aggiunta parte di modulo AI Assistant	Luca Slongo	-	-
0.1.2	31/03/2026	Prima stesura parte front-end	Leonardo Salviato	-	-
0.1.1	31/03/2026	Continuazione paragrafo Pattern e aggiunta diagrammi	Marco Favero	-	-
0.1.0	30/03/2026	Verifica del documento	-	Alberto Autiero	-
0.0.5	27/03/2026	Prima stesura paragrafo Database e continuazione paragrafo Design Pattern	Leonardo Salviato	-	-
0.0.4	25/03/2026	Stesura paragrafo Tecnologie e Linguaggi	Leonardo Salviato	-	-
0.0.3	18/03/2026	Prima stesura design pattern	Leonardo Salviato	-	-
0.0.2	16/03/2026	Prima stesura della tabella dei requisiti, paragrafo requisiti e paragrafo architettura	Leonardo Salviato	-	-
0.0.1	24/02/2026	Prima stesura del documento	Alberto Pignat	-	-

Indice

1 Introduzione **6**
1.1 Glossario . . . 6
1.1.1 Riferimenti normativi . . . 6

2 Tecnologie **6**
2.1 Linguaggi . . . 6
2.2 Framework . . . 7
2.3 Librerie . . . 7
2.4 Strumenti di Sviluppo . . . 8

3 API **9**
3.1 Document Management API . . . 9
3.2 Lookups API . . . 9
3.3 Sendings API . . . 10
3.4 Templates API . . . 10
3.5 Tones & Styles API . . . 10
3.6 Posts API . . . 10
3.7 Generated Data API . . . 11
3.8 Analytics API . . . 11

4 Architettura del Sistema **11**
4.1 Contesto di Sistema (C4 Level 1) . . . 11
4.2 Architettura Applicativa (C4 Level 2 - Container) . . . 12
4.3 Architettura Logica Interna (Layered Architecture) . . . 13
4.3.1 Livello di Presentazione (Presentation Layer) . . . 13
4.3.2 Livello applicativo e di Business Logic (Application Layer) . . . 13
4.3.3 Livello di Accesso ai Dati (Data Access Layer) . . . 14
4.3.4 Livello Dati (Data Layer) . . . 14
4.4 Dettaglio dei Componenti (C4 Level 3) . . . 14
4.4.1 Modulo OCR ed Estrazione Dati: . . . 14
4.4.2 Modulo Generazione Contenuti: . . . 15
4.4.3 Modulo Analisi e Reportistica (Data Analyst): . . . 16
4.5 Architettura di Deployment (Monolite containerizzato) . . . 17

5 Descrizione architettura (Backend) **18**
5.1 Modulo AI Assistant . . . 18
5.1.1 Diagrammi classi parziali del modulo AI Assistant . . . 18
5.2 Modulo AI CoPilot . . . 28
5.2.1 Diagrammi classi parziali del blocco OCR . . . 29
5.2.2 Diagrammi di sequenza del blocco OCR . . . 33
5.3 Modulo Data analisi . . . 57
5.3.1 Diagramma delle classi blocco modulo Data analisi . . . 58
5.4 Classi condivise . . . 62
5.4.1 Diagramma delle classi blocco modulo Classi condivise . . . 63

6 Descrizione architettura (Frontend) **66**

7 Design Pattern **66**
7.1 Pattern implementati esplicitamente . . . 67
7.1.1 Service Object . . . 67

7.1.2 Command . . . 69
7.1.3 Repository . . . 70
7.1.4 Container e Dependency Injection . . . 72
7.1.5 Strategy . . . 74
7.1.6 Factory Method . . . 75
7.1.7 Presenter e Serializer . . . 76
7.1.8 Observer (ActionCable Notifier) . . . 77
7.2 Pattern ereditati dal framework . . . 79
7.2.1 Model-View-Controller (MVC) . . . 79
7.2.2 Active Record . . . 80
7.2.3 Observer tramite Callbacks . . . 81
7.3 Database . . . 82
7.3.1 Schema ER . . . 82
7.3.2 Schema ERD (Entity-Relationship Diagram) . . . 82
7.3.3 Descrizione tabelle . . . 83

8 Requisiti Funzionali Soddisfatti 85
8.1 Completamento requisiti . . . 85
8.2 Tabella di tracciamento dei requisiti . . . 85

Elenco delle tabelle

3 Tabella dei Linguaggi . . . 6
4 Tabella dei Framework . . . 7
5 Tabella delle Librerie . . . 7
6 Tabella degli Strumenti di Sviluppo . . . 8
7 Document Management API . . . 9
8 Lookups API . . . 10
9 Sendings API . . . 10
10 Templates API . . . 10
11 Tones & Styles API . . . 10
12 Posts API . . . 10
13 Generated Data API . . . 11
14 Analytics API . . . 11
16 Riepilogo dei design pattern adottati . . . 67
18 Command e rispettivi job accodati . . . 70
19 Tabella dei Requisiti funzionali$_G$ . . . 86

Elenco delle figure

1 System Context Diagram . . . 12
2 Container Diagram . . . 13
3 Component Diagram – Document Processing . . . 15
4 Component Diagram – AI Generation . . . 16
5 Component Diagram – Data Analysis . . . 17
6 Architettura di Deployment . . . 18
7 Diagramma classi parziale – AIGeneratorController . . . 18
8 Diagramma classi parziale – TonesController . . . 19
9 Diagramma classi parziale – StylesController . . . 19
10 Diagramma classi parziale – PostsController . . . 19

11 Diagramma classi parziale – Document Controller 29
12 Diagramma classi parziale – Flusso di split PDF 30
13 Diagramma classi parziale – Flusso di estrazione da PDF 31
14 Diagramma classi parziale – Flusso file generici 32
15 Diagramma di sequenza – Flusso Generic File 33
16 Diagramma di sequenza – Flusso PDF/OCR 34
17 Schema ERD1 82
18 Schema ERD2 83

1 Introduzione

Il documento ha come scopo la descrizione dettagliata dell'architettura del sistema, delle tecnologie utilizzate e delle API esposte. Viene inoltre fornita una panoramica dei requisiti funzionali soddisfatti dal prodotto software.

1.1 Glossario

Il glossario$_G$ raccoglie e definisce i termini, gli acronimi e le abbreviazioni impiegati nel documento e nel progetto$_G$ Nexum. L'obiettivo è fornire definizioni univoche per ridurre ambiguità, garantire coerenza terminologica tra i membri del team e facilitare l'onboarding di nuovi partecipanti.
Per i termini tecnici e specifici utilizzati in questo documento, si fa riferimento al glossario$_G$ disponibile al seguente link. Per maggiore usabilità e facilitá di consultazione, il glossario$_G$ è accessibile anche aprendo dal nostro sito web i vari documenti con il viewer pdf da noi sviluppato.

1.1.1 Riferimenti normativi

- **Capitolato$_G$ d'appalto C5: Nexum - Piattaforma di consulenza e documentazione previdenziale**
  https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C5.pdf

2 Tecnologie

Il presente capitolo illustra lo stack tecnologico adottato per la progettazione e la realizzazione del prodotto software. Le scelte implementative sono state guidate dalla necessità di sviluppare un'architettura solida, garantendo al contempo elevati standard di scalabilità, manutenibilità e sicurezza del sistema. Nelle sottosezioni seguenti verranno passati in rassegna i linguaggi di programmazione, i framework, le librerie e gli strumenti di supporto operativi impiegati dal team, evidenziando il ruolo specifico di ciascun elemento all'interno del ciclo di vita del progetto.

2.1 Linguaggi

Questa sezione descrive i linguaggi di programmazione, di markup e di interrogazione fondamentali utilizzati per la stesura del codice sorgente. La selezione comprende sia gli standard consolidati per la strutturazione e stilizzazione dell'interfaccia utente (Frontend), sia i linguaggi orientati agli oggetti scelti per implementare la logica applicativa (Backend), fino ad arrivare agli strumenti dichiarativi per la gestione delle basi di dati. La Tabella 3 riassume i linguaggi adottati, specificandone la versione di riferimento e il relativo ambito di utilizzo.

Tabella 3: Tabella dei Linguaggi

Nome	Descrizione	Versione
Typescript	Linguaggio di programmazione basato su JavaScript che aggiunge la tipizzazione statica	6.0
HTML	Linguaggio di markup usato per creare pagine web	5

Continua nella prossima pagina...

Tabella 3 – continua dalla pagina precedente

Nome	Descrizione	Versione
CSS	Linguaggio usato per definire lo stile delle pagine web	3
Ruby	Linguaggio di programmazione orientato agli oggetti	3.3.1
SQL	Linguaggio per la gestione e manipolazione di database relazionali	SQL:2023

2.2 Framework

In questa sezione vengono presentati i framework adottati per costituire l'impalcatura architetturale del prodotto software. La scelta si è orientata verso soluzioni moderne e ampiamente supportate dalla community, capaci di garantire una netta separazione tra la logica di presentazione e le regole di business. Nello specifico, la Tabella 4 illustra le infrastrutture cardine impiegate per lo sviluppo della Single Page Application lato client e per la realizzazione delle API lato server.

Tabella 4: Tabella dei Framework

Nome	Descrizione	Versione
Angular	Framework open source per la creazione e sviluppo di single page applications	21
Ruby on Rails	Framework open source server side per applicazioni web	8.1

2.3 Librerie

Oltre alle tecnologie di base, lo sviluppo del sistema ha richiesto l'integrazione di pacchetti e librerie di terze parti. Queste componenti si sono rivelate essenziali per estendere le funzionalità native dei framework, ottimizzare i tempi di sviluppo e risolvere specifiche problematiche di dominio in modo standardizzato, affidabile e sicuro. La Tabella 5 elenca le principali dipendenze esterne introdotte all'interno degli ambienti Node.js e Ruby.

Tabella 5: Tabella delle Librerie

Nome	Descrizione	Versione
rails	Framework web full-stack per Ruby.	8.1.2.1
pg	Adattatore PostgreSQL per Active Record.	1.1
puma	Server web concorrente, veloce e multi-thread per Ruby.	5.0.0
tzinfo-data	Dati sui fusi orari necessari per i sistemi Windows e JRuby.	1.2026.1
solid_cache	Backend basato su database per Rails.cache.	1.0.10
solid_queue	Backend basato su database per Active Job.	1.4.0
solid_cable	Backend basato su database per Action Cable.	3.0.12
bootsnap	Ottimizza e riduce i tempi di avvio dell'applicazione tramite caching.	1.23.0
kamal	Strumento per il deploy di applicazioni web ovunque tramite container Docker.	2.11.0

Continua nella prossima pagina...

Tabella 5 – continua dalla pagina precedente

Nome	Descrizione	Versione
thruster	Proxy HTTP per accelerare Puma con caching degli asset e compressione.	0.1.20
image_processing	Fornisce metodi helper per la gestione e la trasformazione delle immagini.	1.2.0
rack-cors	Middleware per la gestione delle richieste CORS (Cross-Origin Resource Sharing).	3.0.0
combine_pdf	Libreria per unire, formattare o manipolare file PDF.	1.0.31
csv	Fornisce un'interfaccia standard per leggere e scrivere file CSV.	3.3.5
aws-sdk-textract	Client AWS SDK ufficiale per il servizio OCR Amazon Textract.	1.90.0
aws-sdk-bedrockruntime	Client AWS SDK ufficiale per interagire con i modelli LLM di Amazon Bedrock.	1.74.0
simplecov	Strumento per l'analisi della copertura del codice durante i test.	0.22.0
minitest	Suite completa e leggera per il testing (TDD, BDD, benchmarking).	6.0.3
minitest-mock	Estensione per Minitest dedicata al mocking di oggetti nei test.	5.27.0
mocha	Libreria ampiamente utilizzata per il mocking e lo stubbing nei test.	3.1.0
debug	Strumento standard per il debugging delle applicazioni Ruby.	1.11.1
bundler-audit	Scanner per rilevare vulnerabilità di sicurezza note nelle gemme installate.	0.9.3
brakeman	Scanner di analisi statica per rilevare vulnerabilità di sicurezza in app Rails.	8.0.4
rubocop-rails-omakase	Configurazione standard ufficiale (stile Omakase) per RuboCop in Rails.	1.1.0

2.4 Strumenti di Sviluppo

L'efficienza, la solidità e la riproducibilità dell'intero ciclo di vita del software sono state assicurate attraverso l'impiego di una serie di strumenti operativi e di orchestrazione avanzati. Questa categoria, dettagliata nella Tabella 6, comprende gli ambienti di sviluppo integrati (IDE), i sistemi di containerizzazione volti a uniformare gli ambienti tra i vari membri del team, e le piattaforme per la modellazione e l'amministrazione della persistenza dei dati.

Tabella 6: Tabella degli Strumenti di Sviluppo

Nome	Descrizione	Versione
Visual Studio Code	IDE open source per lo sviluppo software sviluppato da Microsoft	1.114
Docker	Piattaforma che permette di creare, distribuire e eseguire applicazioni in container	29.3.1
PostgreSQL	Database relazionale open source	18.3

Continua nella prossima pagina...

Tabella 6 – continua dalla pagina precedente

Nome	Descrizione	Versione
pgAdmin	Strumento grafico per amministrare e gestire database PostgreSQL	9.13

3 API

Il sistema espone un set di API RESTful che consentono l'interazione tra il frontend e il backend, nonché l'integrazione con eventuali servizi esterni. Le API sono progettate per essere intuitive, sicure e performanti, seguendo le best practice del settore per la progettazione di interfacce di programmazione. Di seguito viene fornita una panoramica delle principali API esposte dal sistema, suddivise per funzionalità e endpoint.

3.1 Document Management API

Tabella 7: Document Management API

Endpoint	Metodo	Descrizione
/documents/split	POST	Upload PDF, avvia splitting + estrazione
/documents/process_-file	POST	Upload CSV/immagine
/documents/uploads	GET	Lista documenti caricati
/documents/uploads/:id/extracted	GET	Pagine estratte di un documento
/documents/uploads/:id/file	GET	Download file originale
/documents/extracted/:id	GET	Dettaglio documento estratto
/documents/extracted/:id/pdf	GET	Download range pagine come PDF
/documents/extracted/:id/metadata	PATCH	Aggiorna metadati estratti
/documents/extracted/:id/validate	PATCH	Valida documento
/documents/extracted/:id/reassign_range	PATCH	Riassegna range pagine estratto
/documents/uploads/:id/retry	POST	Riprova processing run fallito
/documents/extracted/:id/retry	POST	Riprova estrazione fallita
/documents/uploads/:id	DELETE	Elimina documento caricato

3.2 Lookups API

Tabella 8: Lookups API

Endpoint	Metodo	Descrizione
/lookups/companies	GET	Nomi aziende
/lookups/users	GET	Nomi dipendenti

3.3 Sendings API

Tabella 9: Sendings API

Endpoint	Metodo	Descrizione
/sendings	GET	Lista invii effettuati
/sendings	POST	Crea un invio (documento → destinatario)

3.4 Templates API

Tabella 10: Templates API

Endpoint	Metodo	Descrizione
/templates	GET	Lista template disponibili
/templates/ :id	GET	Dettaglio template
/templates	POST	Crea nuovo template

3.5 Tones & Styles API

Tabella 11: Tones & Styles API

Endpoint	Metodo	Descrizione
/tones	GET	Lista toni disponibili
/tones	POST	Crea tono
/tones/:id	DELETE	Elimina tono
/styles	GET	Lista stili disponibili
/styles	POST	Crea stile
/styles/:id	DELETE	Elimina stile

3.6 Posts API

Tabella 12: Posts API

Endpoint	Metodo	Descrizione
/posts	GET	Lista post generati
/posts	POST	Crea post
/posts/:id	DELETE	Elimina post

3.7 Generated Data API

Tabella 13: Generated Data API

Endpoint	Metodo	Descrizione
/generated_data	POST	Crea contenuto AI (asincrono)
/generated_data/:id/regenerate	POST	Rigenera contenuto AI
/generated_data/:id	GET	Stato e contenuto generazione
/generated_data/:id/rating	PATCH	Valuta contenuto generato
/generated_data/:id	DELETE	Elimina generazione

3.8 Analytics API

Tabella 14: Analytics API

Endpoint	Metodo	Descrizione
/ai_copilot_data_analyst	GET	Dati analisi CoPilot
/ai_generator_data_analyst	GET	Dati analisi Generator

4 Architettura del Sistema

Il sistema è stato documentato seguendo un approccio top-down basato sul modello C4, partendo dal contesto generale fino ad arrivare al dettaglio dei componenti interni e all'infrastruttura di rilascio.

4.1 Contesto di Sistema (C4 Level 1)

Il diagramma di contesto illustra le interazioni di altissimo livello tra l'utente finale e i sistemi esterni. La Piattaforma MVP si posiziona al centro, fornendo all'Operatore funzionalità di caricamento documenti, validazione estrazioni e generazione contenuti tramite browser (via HTTPS e WebSocket). Per l'elaborazione dei dati, il sistema si integra con l'ecosistema cloud di Amazon Web Services, delegando l'estrazione ottica dei caratteri (OCR) ad AWS Textract e l'inferenza di intelligenza artificiale ad AWS Bedrock.

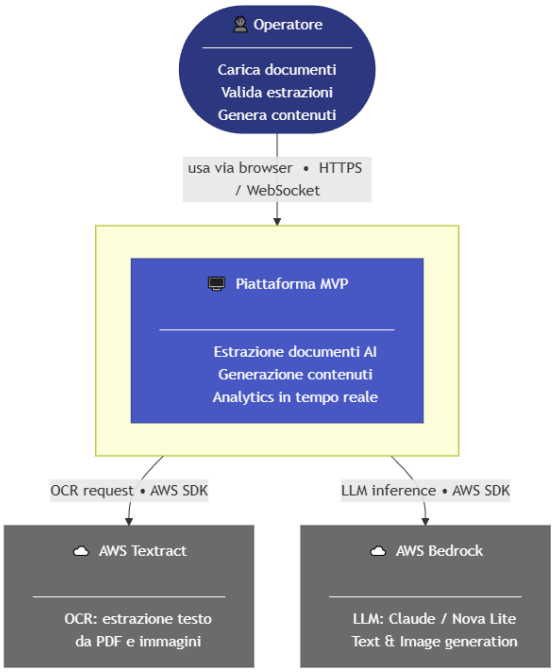


Figura 1: System Context Diagram

4.2 Architettura Applicativa (C4 Level 2 - Container)

Scendendo al livello applicativo, il sistema si divide nei suoi container logici principali. Il frontend è costituito da una Single Page Application (SPA) sviluppata in Angular 21. Il backend è un'API sviluppata in Ruby on Rails 8, servita tramite Puma. Per garantire fluidità all'utente, le operazioni pesanti (come l'OCR e l'invocazione degli LLM) sono gestite in modo asincrono tramite Solid Queue Workers. Il livello di persistenza è unificato su PostgreSQL 16 (Solid Stack), che funge non solo da database primario, ma anche da gestore per le code, cache e backend per le connessioni WebSocket (ActionCable).

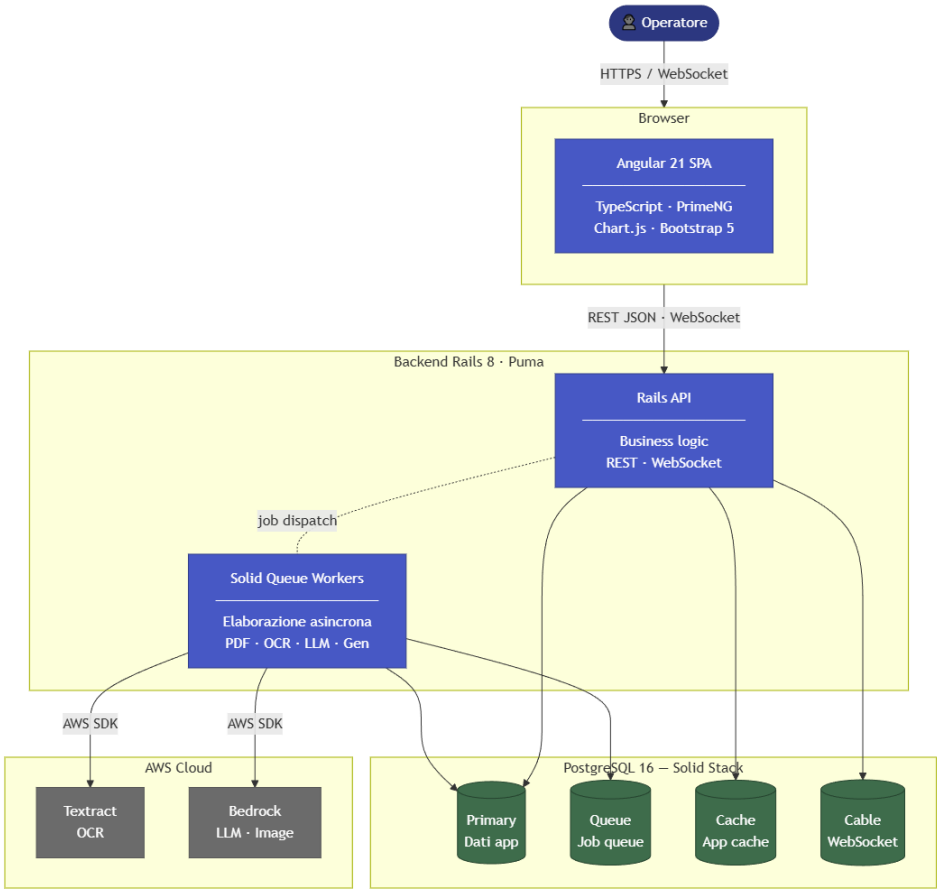


Figura 2: Container Diagram

4.3 Architettura Logica Interna (Layered Architecture)

Il sistema è stato progettato seguendo un pattern architetturale di tipo Layered (a livelli). Questa separazione garantisce un alto grado di disaccoppiamento, facilitando la manuntenzione, i test e la scalabilità del sistema. Inoltre, lavorando in ruby on rails, è stato possibile sfruttare le convenzioni del framework per implementare in modo naturale i vari livelli, senza andare a forzare l'introduzione di ulteriori astrazioni o pattern complessi e contro il linguaggio stesso. L'architettura logica si divide in quattro livelli principali:

4.3.1 Livello di Presentazione (Presentation Layer)

Sviluppato in Angular (Single Page Application), è responsabile dell'interfaccia utente e dell'interazione con l'utente finale. Comunica con il backend tramite API RESTful.

- Riceve gli input dell'utente (es. caricamento documenti, richieste di generazione testo)
- Comunica con il backend per inviare richieste e ricevere risposte.
- Gestisce il routing lato client e la renderizzazione reattiva dei dati.

4.3.2 Livello applicativo e di Business Logic (Application Layer)

Sviluppato in Ruby on Rails (API-only). Rappresenta il cuore del sistema e si struttura internamente in ulteriori sotto-livelli:

- **Controller**: gestisce le richieste HTTP, orchestrando le operazioni necessarie per soddisfarle.
- **Service**: contiene la logica di business specifica, come l'integrazione con i modelli di intelligenza artificiale, la gestione dei documenti e la logica di elaborazione.
- **Model**: rappresenta le entità del dominio (es. Utente, Documento, Sessione) e gestisce la persistenza dei dati.

4.3.3 Livello di Accesso ai Dati (Data Access Layer)

Gestito dal modulo ActiveRecord di Ruby on Rails (ORM), si occupa della comunicazione con il database PostgreSQL. Fornisce un'interfaccia per eseguire operazioni CRUD (Create, Read, Update, Delete) sui dati persistenti traducendo le operazioni in query SQL sicure e ottimizzate, prevenendo vulnerabilità come le SQL Injection.

4.3.4 Livello Dati (Data Layer)

Costituito dal Database Relazionale (RDBMS) PostgreSQL. Ha la responsabilità di memorizzare in modo persistente tutte le informazioni necessarie al funzionamento del sistema, inclusi dati degli utenti, documenti caricati, sessioni di interazione e log delle attività.

4.4 Dettaglio dei Componenti (C4 Level 3)

I principi dell'architettura Layered appena descritti trovano applicazione pratica nell'organizzazione interna dei macro-moduli del backend Rails. Di seguito si illustra il dettaglio (Livello 3) dei tre domini principali del sistema:

4.4.1 Modulo OCR ed Estrazione Dati:

Questo modulo gestisce il caricamento dei file, lo splitting asincrono dei PDF e l'orchestrazione con AWS Textract e Bedrock. Si nota la netta separazione tra il livello HTTP (Controllers), i Commands per l'accodamento dei job, gli Orchestratori e i Domain Services (Adapter per le API esterne e logica di validazione).

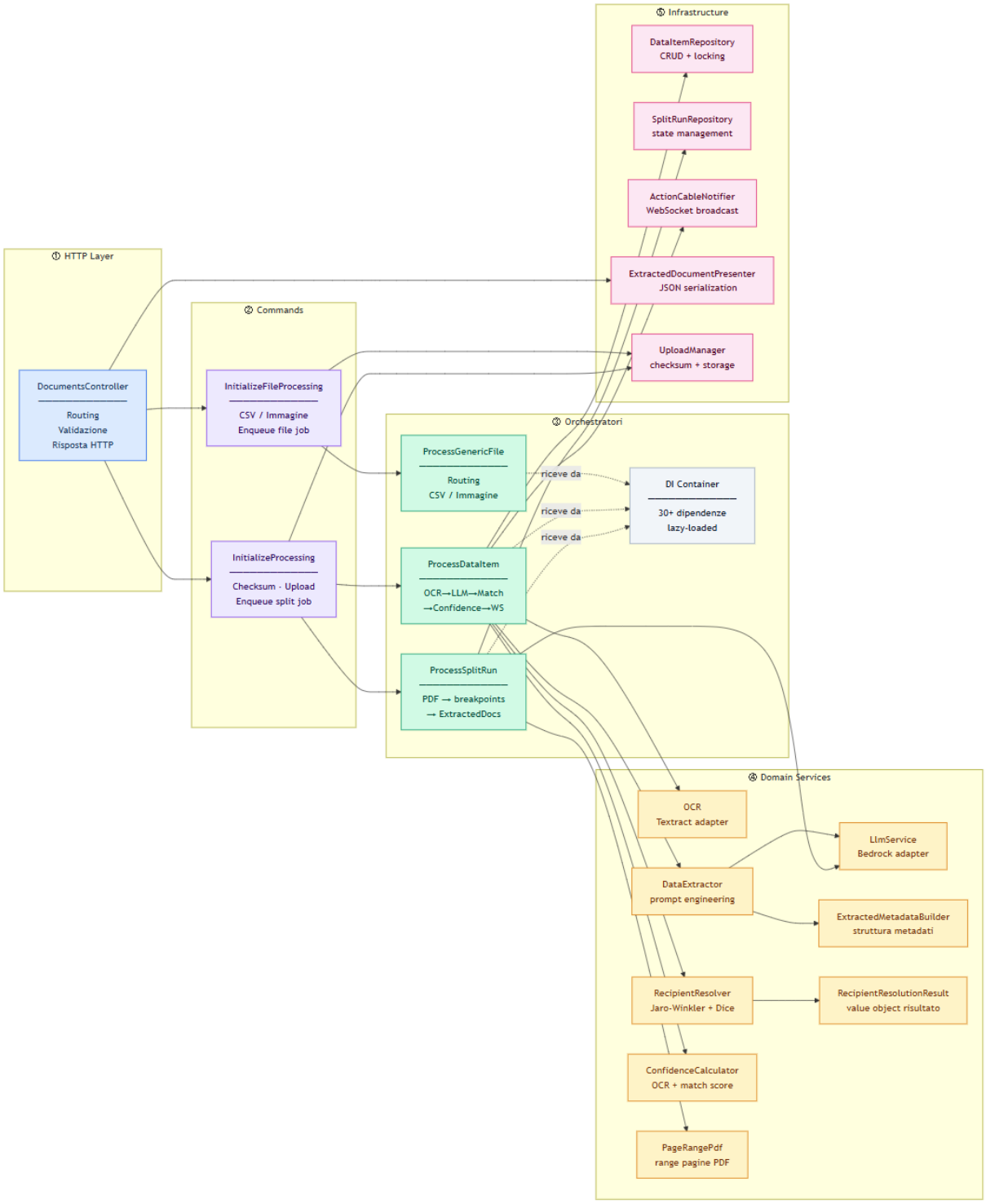


Figura 3: Component Diagram – Document Processing

4.4.2 Modulo Generazione Contenuti:

Gestisce la pipeline di generazione multimodale (testo e immagini). Il controller delega immediatamente l'esecuzione a un job asincrono (AIGeneratorJob), il quale coordina i servizi di generazione testuale (Bedrock) e visiva, salvando infine i record generati tramite i Data Manager.

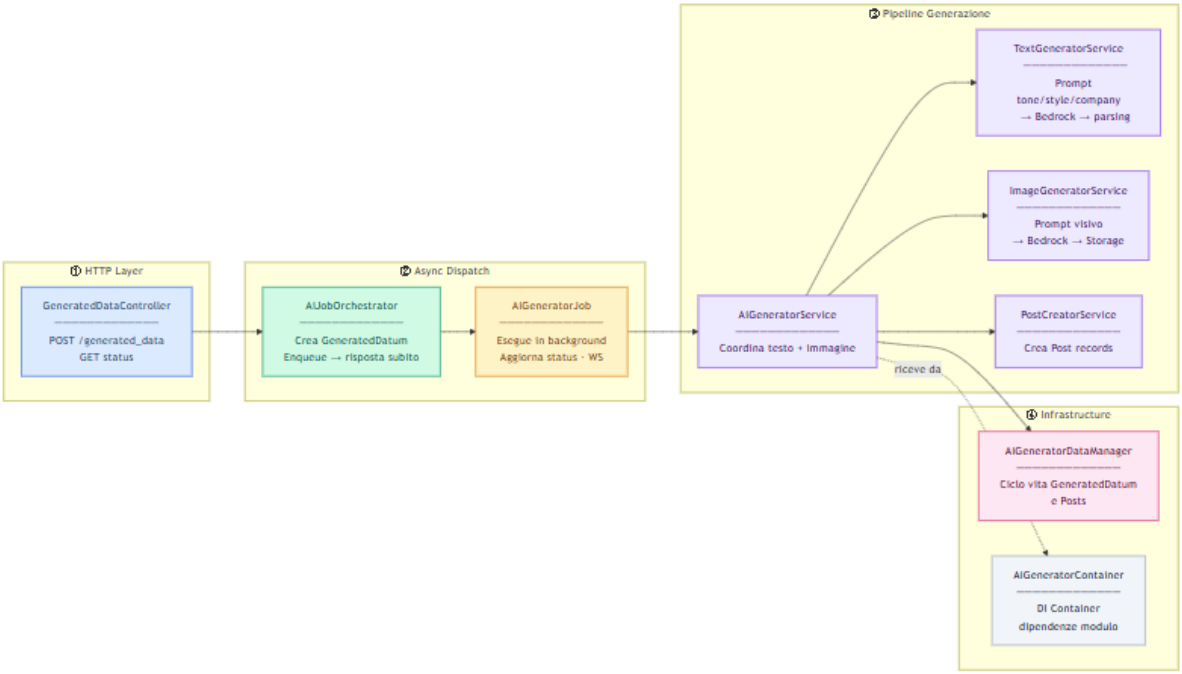


Figura 4: Component Diagram – AI Generation

4.4.3 Modulo Analisi e Reportistica (Data Analyst):

Questo componente si occupa dell'aggregazione dei dati per le dashboard. L'architettura fa uso del pattern Template Method nei *Computation Services* e demanda l'esecuzione delle query aggregate (es. calcolo della confidence media o dei tempi di elaborazione) a specifici *Data Managers*.

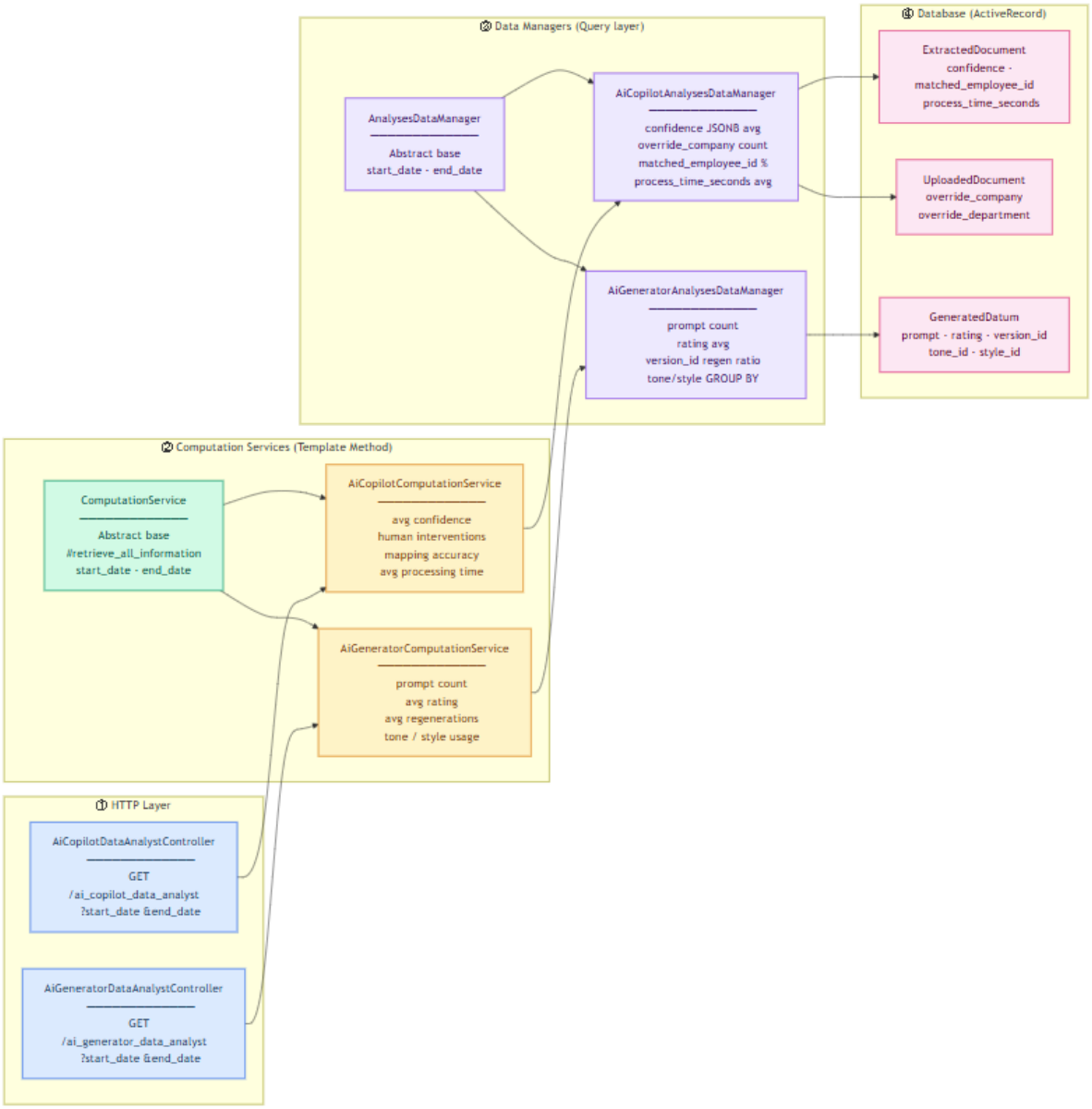


Figura 5: Component Diagram – Data Analysis

4.5 Architettura di Deployment (Monolite containerizzato)

Sebbene logicamente il sistema sia diviso in Frontend e Backend (decoupled), dal punto di vista del rilascio in produzione si è optato per un approccio monolitico basato su container. Tutte le componenti del sistema sono pacchettizzate e distribuite insieme come un'unica entità logica su un singolo nodo ospite (Server/Macchina Virtuale), garantendo un ambiente isolato, riproducibile e coerente dalla fase di sviluppo fino alla produzione. L'orchestrazione locale è affidata a Docker Compose.

- **Docker Host (Server Node)**: L'infrastruttura hardware o virtuale principale che ospita il motore Docker. Al suo interno risiede una Rete Virtuale Privata (Docker Network) che permette ai container di comunicare tra loro in totale sicurezza, senza esporre porte non necessarie all'esterno.

- **Container Frontend (Web Server)**: Un container dedicato all'esecuzione dell'applicativo Angular. Nelle richieste esterne, questo nodo funge da punto di ingresso per gli utenti, servendo i file statici della Single Page Application.
- **Container Backend (Application Server)**: Un container isolato basato su immagine Ruby che esegue il server web di Rails. È esposto internamente per ricevere le chiamate API dal frontend e processare la logica di business.
- **Container Database (Database Server)**: Un container basato sull'immagine ufficiale di PostgreSQL. Persiste i dati fisicamente scrivendoli su un Docker Volume esterno al container stesso, garantendo che i dati del sistema non vadano persi in caso di riavvio o aggiornamento del container.

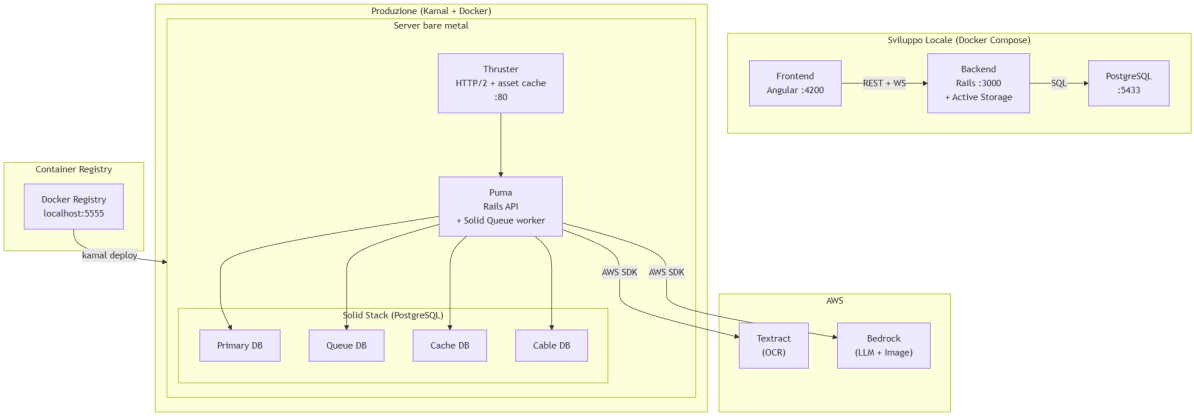


Figura 6: Architettura di Deployment

5 Descrizione architettura (Backend)

5.1 Modulo AI Assistant

Il modulo AI Assistant è la componente del sistema dedicata alla generazione di contenuti testuali e visivi tramite modelli di intelligenza artificiale. Il suo obiettivo è fornire agli utenti uno strumento potente e intuitivo per creare post personalizzati, arricchendo l'esperienza utente con funzionalità avanzate di generazione automatica di contenuti. Il modulo si integra con i servizi di AI (AWS Bedrock) per estrarre il massimo valore dalle capacità di generazione testuale e visiva, offrendo un'interfaccia semplice per la creazione di contenuti complessi e personalizzati, supportando al contempo la gestione e la valutazione dei risultati generati. Nei diagrammi seguenti sono rappresentati i componenti del modulo, divisi per controller, evidenziando i componenti principali e le loro interazioni.

Il controller più imporante del modulo è *AIGeneratorController*, che espone gli endpoint per la creazione e gestione dei contenuti generati. Il controller delega la logica di business a un job asincrono *(AIGeneratorJob)*, che si occupa di orchestrare le chiamate ai servizi di generazione testuale e visiva, gestire la persistenza dei risultati e aggiornare lo stato delle generazioni in corso.

5.1.1 Diagrammi classi parziali del modulo AI Assistant

Figura 7: Diagramma classi parziale – AIGeneratorController

Il diagramma mostra il controller di ingresso del modulo: riceve le richieste HTTP, valida i parametri e delega l'esecuzione al job asincrono. Tramite l'utilizzo di un container di dipendenza, il controller inietta i servizi necessari per la generazione, mantenendo un alto grado di disaccoppiamento e testabilità. I vari services gestiscono la logica di interazione con i modelli di intelligenza artificiale, la persistenza dei dati e l'aggiornamento dello stato delle generazioni.

Figura 8: Diagramma classi parziale – TonesController

Il diagramma mostra il controller dedicato alla gestione dei toni di generazione. Espone endpoint per la creazione, eliminazione e recupero dei toni disponibili, delegando la logica di business al servizio specifico che si occupa della persistenza e della validazione dei dati: *ToneSerializer*.

Figura 9: Diagramma classi parziale – StylesController

Il diagramma mostra il controller dedicato alla gestione dei stili di generazione. Il funzionamento è analogo a quello del controller dei toni, delegando la logica di business al servizio specifico *StyleSerializer*.

Figura 10: Diagramma classi parziale – PostsController

Il diagramma mostra il controller dedicato alla pubblicazione di post. Anche qui il funzionamento di base è analogo a quello del controller dei toni, fatto utilizzando *PostsSerializer*, ma utilizza anche *PostCreatorService* per gestire la logica di creazione dei post, con la sua logica specifica di validazione.

5.1.1.1 AIGeneratorController

Classe di tipo Controller che espone gli endpoint per la generazione dei contenuti nel modulo AI Assistant. Gestisce le operazioni CRUD sulle generazioni, espone un endpoint per rigenerare i contenuti e un endpoint per valutare i risultati generati.

Attributi

- Nessuno

Metodi

- create(params: Hash): Json
- regenerate(id: int): Json
- show(id: int): Json
- rating(id: int, rating: int): Json
- destroy(id: int): Json
- generation_params(): Hash

5.1.1.2 AIJobOrchestrator

Classe orchestratrice che gestisce la logica di esecuzione dei job di generazione dei contenuti. Si occupa di coordinare le chiamate ai servizi di generazione testuale e visiva, gestire la persistenza dei risultati e aggiornare lo stato delle generazioni in corso.

Attributi

- Nessuno

Metodi

- orchestrate(params: Hash): int
- signal_process_started(generationId: int): void
- signal_failure(generation_id: int, error: string): void
- complete(generationId: int): void
- ensure_active_configuration(params: Hash): bool

5.1.1.3 ActionCable

Classe nativa di rails per la gestione delle connessioni WebSocket. Nel contesto del modulo AI Assistant, viene utilizzata per inviare aggiornamenti in tempo reale al frontend riguardo lo stato delle generazioni in corso, permettendo un'esperienza utente più fluida e interattiva.

Attributi

- Nessuno

Metodi

- broadcast(channel: string, message: Json): void

5.1.1.4 AIGeneratorJob

Classe di tipo Job asincrono che si occupa di eseguire la logica di generazione dei contenuti in background. Viene invocata dal *AIJobOrchestrator* quando viene richiesta la creazione o rigenerazione di un contenuto, e si occupa di orchestrare le chiamate ai servizi di generazione testuale e visiva.

Attributi

- Nessuno

Metodi

- perform(generationId: int): void

5.1.1.5 AIGeneratorContainer

Container che si occupa di gestire le dipendenze necessarie per l'esecuzione dei job di generazione dei contenuti. Viene utilizzato per iniettare i servizi necessari all'esecuzione dei job all'interno di *AIGeneratorService*, mantenendo un alto grado di disaccoppiamento e testabilità, ritornando alla chiamata del job i servizi necessari per l'esecuzione della logica di generazione.

Attributi

- aiGeneratorService
- imageGeneratorService
- setterFactory
- textGeneratorService
- aiGeneratorDataManager
- textResponseValidator

Metodi

- initialize(): void
- aiGeneratorService(): AIGeneratorService
- imageGeneratorService(): ImageGeneratorService
- setterFactory(): SetterFactory
- textGeneratorService(): TextGeneratorService
- aiGeneratorDataManager(): AIGeneratorDataManager
- textResponseValidator(): TextResponseValidator

5.1.1.6 AIGeneratorService

Classe orchestratrice che gestisce la logica di esecuzione dei job di generazione dei contenuti. Si occupa di coordinare le chiamate ai servizi di generazione testuale e visiva, gestire la persistenza dei risultati e aggiornare lo stato delle generazioni in corso.

Attributi

- imgGenerator
- textGenerator
- aiGeneratorDataManager
- textResponseValidator
- setterFactory

Metodi

- initialize(imgGenerator: ImageGeneratorService, textGenerator: TextGeneratorService, aiGeneratorDataManager: AIGeneratorDataManager, textResponseValidator: TextResponseValidator, setterFactory: SetterFactory): void
- createContent(generationId: int): int
- createText(textSetter: TextParamsSetterService, userPrompt: string): string
- createImage(imageSetter: ImageParamsSetterService, textResult: string): Hash
- validateSetters(textSetter: TextParamsSetterService, imageSetter: ImageParamsSetterService): bool
- prefixed_errors(prefix: string, errors: string[]): string[]

5.1.1.7 AIGeneratorDataManager

Classe di tipo Data Manager che si occupa della gestione dei dati relativi alle generazioni dei contenuti. Si occupa di salvare i risultati generati, aggiornare lo stato delle generazioni in corso e fornire un'interfaccia per recuperare i dati delle generazioni quando necessario.

Attributi

- Nessuno

Metodi

- fetchCompanyDescription(companyId: int): string
- fetchToneDescription(toneId: int): string
- fetchStyleDescription(styleId: int): string
- fetchGenerationData(generationId: int): GeneratedDatum
- saveContent(generationId: int, responseData: Hash): int

5.1.1.8 SetterFactory

Classe di tipo Factory che si occupa di creare i setter per i parametri di generazione dei contenuti. Viene utilizzata per creare i setter specifici per i parametri di testo e immagine, in base alle esigenze della generazione richiesta.

Attributi

- Nessuno

Metodi

- createTextSetter(params: Hash): TextParamsSetterService
- createImageSetter(params: Hash): ImageParamsSetterService

5.1.1.9 TextParamsSetterService

Classe Service che si occupa di impostare i parametri di generazione per i contenuti testuali. Viene utilizzata per impostare i parametri specifici per la generazione di testo,come il tono, lo stile e la descrizione dell'azienda, in base alle esigenze della generazione richiesta.

Attributi

- prompt: string
- toneDescription: string
- styleDescription: string
- companyName: string
- companyDescription: string
- errors: string[]

Metodi

- initialize(params: Hash): void
- valid(): bool
- buildSystemPrompt(): string
- getData(): Hash

5.1.1.10 ImageParamsSetterService

Classe Service che si occupa di impostare i parametri di generazione per i contenuti immagine. Viene utilizzata per impostare i parametri specifici per la generazione di immagini, in base alle esigenze della generazione richiesta, utilizzando anche la risposta della generazione testuale per ulteriori dettagli.

Attributi

- prompt: string
- width: int
- height: int
- seed: string
- errors: string[]

Metodi

- initialize(params: Hash): void
- valid(): bool
- buildImagePrompt(aiTextGeneratorResponse: string): Json
- getData(): Hash

5.1.1.11 TextGeneratorService

Classe service che si occupa di gestire la logica di generazione dei contenuti testuali. Si occupa di interagire con il servizio di generazione testuale (AWS Bedrock) per generare i contenuti richiesti, utilizzando i parametri impostati dal *TextParamsSetterService*.

Attributi

- region: string
- client: Aws::BedrockRuntime::Client

Metodi

- initialize(region: string): void
- generateText(user_prompt: string, system_prompt: string): string
- invokeBedrock(modelId: string, user_prompt: string, system_prompt: string, max_tokens: int, temperature: float): AWS::BedrockRuntime::Types::ConverseResponse
- expired_credentials_error?(error: string): string
- expired_credentials_error(error: StandardError, service: Symbol): string

5.1.1.12 ImageGeneratorService

Classe service che si occupa di gestire la logica di generazione dei contenuti immagine. Si occupa di interagire con il servizio di generazione immagini (AWS Bedrock) per generare i contenuti richiesti, utilizzando i parametri impostati dal *ImageParamsSetterService*.

Attributi

- region: string
- client: Aws::BedrockRuntime::Client

Metodi

- initialize(region: string): void
- generateImg(user_prompt: string, system_prompt: string): string
- invokeNovaCanvas(modelId: string, image_prompt_json: Json): AWS::BedrockRuntime::Types::InvokeMode
- extract_image_data(response: AWS::BedrockRuntime::Types::InvokeModelResponse): String
- expired_credentials_error?(error: string): string
- expired_credentials_error(error: StandardError, service: Symbol): string

5.1.1.13 TextResponseValidator

Classe Service che si occupa di validare la risposta della generazione testuale. Si occupa di verificare che la risposta della generazione testuale sia valida e che lo split del titolo dal contenuto sia avvenuto correttamente, restituendo eventuali errori riscontrati durante la validazione.

Attributi

- BLOCKED_RESPONSES: string[]
- TITLE_PREFIXES: string[]

Metodi

- parse(text_result: string, generationId: intx): Hash
- blocked_response_message(text_result: string): string
- split_title_and_content(text_result: string, generationId: int): Hash
- sanitize_title(raw: string): string

5.1.1.14 GeneratedDatumSerializer

Classe serializer che si occupa di serializzare i dati delle generazioni dei contenuti. Viene utilizzata per serializzare i dati delle generazioni in un formato adatto per essere inviato al frontend, gestendo eventuali trasformazioni necessarie per la corretta rappresentazione dei dati.

Attributi

- Nessuno

Metodi

- serialize_collection(generated_data: GeneratedDatum[]): Json
- image_path(generation:int): string
- serialize(generation: int): Json

5.1.1.15 StylesController

Controller dedicato alla gestione degli stili di generazione dei contenuti. Espone endpoint per la creazione, eliminazione e recupero degli stili disponibili.

Attributi

- Nessuno

Metodi

- index(): Json
- create(params: Hash): Json
- destroy(id: int): Json
- style_params(): Hash

5.1.1.16 StylesSerializer

Classe serializer che si occupa di serializzare i dati degli stili di generazione dei contenuti. Viene utilizzata per serializzare i dati degli stili in un formato adatto per essere inviato al frontend, gestendo eventuali trasformazioni necessarie per la corretta rappresentazione dei dati.

Attributi

- Nessuno

Metodi

- serialize_collection(styles: Style[]): Json

5.1.1.17 TonesController

Controller dedicato alla gestione dei toni di generazione dei contenuti. Espone endpoint per la creazione, eliminazione e recupero dei toni disponibili.

Attributi

- Nessuno

Metodi

- index(): Json
- create(params: Hash): Json
- destroy(id: int): Json
- tone_params(): Hash

5.1.1.18 TonesSerializer

Classe serializer che si occupa di serializzare i dati dei toni di generazione dei contenuti. Viene utilizzata per serializzare i dati dei toni in un formato adatto per essere inviato al frontend, gestendo eventuali trasformazioni necessarie per la corretta rappresentazione dei dati.

Attributi

- Nessuno

Metodi

- serialize_collection(tones: Tone[]): Json

5.1.1.19 PostsController

Controller dedicato alla gestione dei post generati. Espone endpoint per la creazione, eliminazione e recupero dei post disponibili, delegando la logica di business alla combinazione di *PostsSerializer* per la serializzazione dei dati e *PostCreatorService* per la logica di creazione dei post, con la sua logica specifica di validazione.

Attributi

- Nessuno

Metodi

- index(): Json
- create(params: Hash): Json
- destroy(id: int): Json
- post_params(): Hash

5.1.1.20 PostsSerializer

Classe serializer che si occupa di serializzare i dati dei post generati. Viene utilizzata per serializzare i dati dei post in un formato adatto per essere inviato al frontend, gestendo eventuali trasformazioni necessarie per la corretta rappresentazione dei dati.

Attributi

- Nessuno

Metodi

- serialize_collection(posts: Post[]): Json

5.1.1.21 PostCreatorService

Classe service che si occupa della creazione dei post. Viene utilizzata per gestire la logica di creazione dei post, inclusa la validazione e la persistenza dei dati. Il servizio si assicura che i dati forniti siano validi e completi prima di creare un nuovo post, restituendo eventuali errori riscontrati durante la validazione.

Attributi

- Nessuno

Metodi

- create_post(params: Hash): Post
- invalid_post(error: string): string

5.2 Modulo AI CoPilot

Il modulo AI CoPilot è la componente del sistema dedicata al supporto dei Consulenti del Lavoro (CdL) nella gestione e revisione documentale automatizzata. L'obiettivo principale è l'efficientamento del workflow operativo attraverso l'impiego di modelli di intelligenza artificiale per l'estrazione di metadati e il riconoscimento anagrafico.

I diagrammi riportati di seguito rappresentano diagrammi delle classi parziali del blocco OCR, utili a mostrare i componenti principali senza appesantire la lettura con l'intero grafo delle dipendenze.

Il primo diagramma mostra *AIDocumentsController*, ossia il controller che gestisce tutto il flusso legato allo split, all'analisi documentale e alle eventuali modifiche manuali (ad esempio validazioni e riassegnazioni). Il controller delega la logica ai servizi applicativi e conclude l'orchestrazione con l'avvio dei job, che innescano asincronamente i flussi rappresentati dagli altri tre diagrammi delle classi parziali. Tutte le dipendenze sono iniettate tramite il container di sistema per massimizzare il disaccoppiamento; il dettaglio completo dell'iniezione non è rappresentato nei diagrammi per ragioni di comodità e chiarezza, ma rimane evidente dall'uso costante di componenti iniettati.

5.2.1 Diagrammi classi parziali del blocco OCR

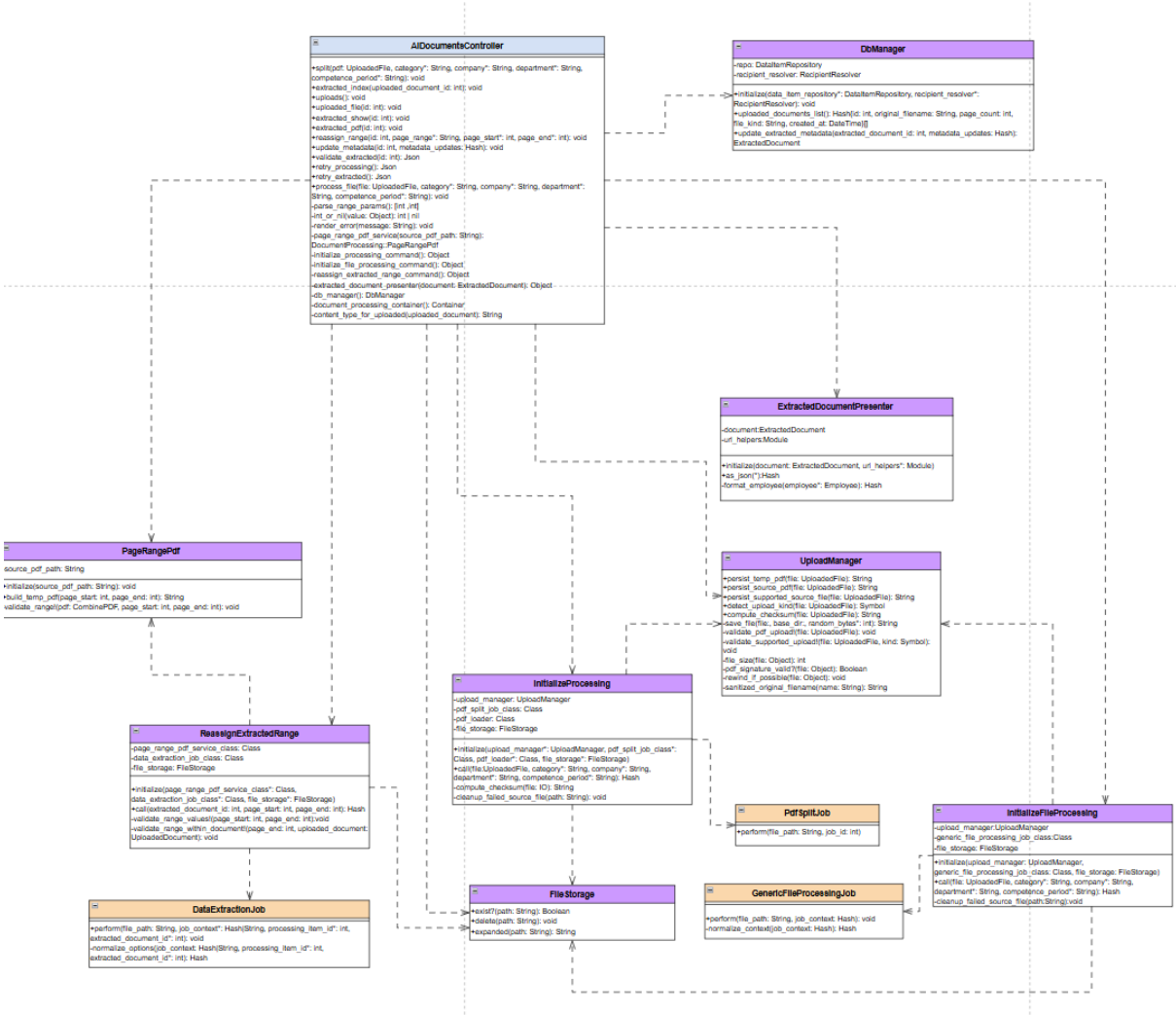


Figura 11: Diagramma classi parziale – Document Controller

Il diagramma mostra il controller di ingresso del modulo OCR: riceve le richieste HTTP, valida i parametri e delega l'esecuzione ai servizi applicativi, fino all'avvio dei job asincroni.

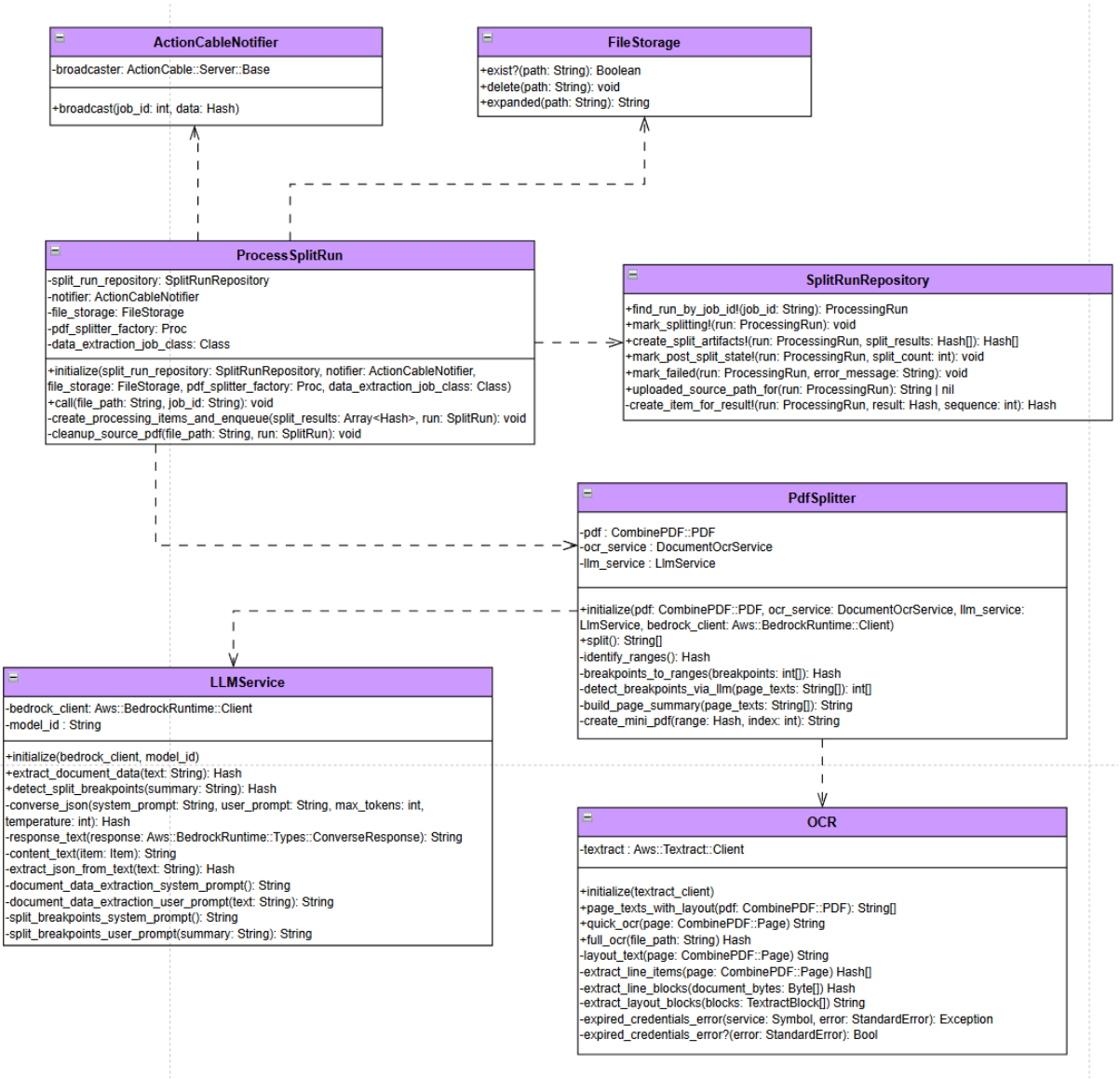


Figura 12: Diagramma classi parziale – Flusso di split PDF

Questo diagramma rappresenta il flusso asincrono di splitting dei PDF, dalla preparazione del file alla suddivisione in item di elaborazione e al dispatch delle estrazioni.

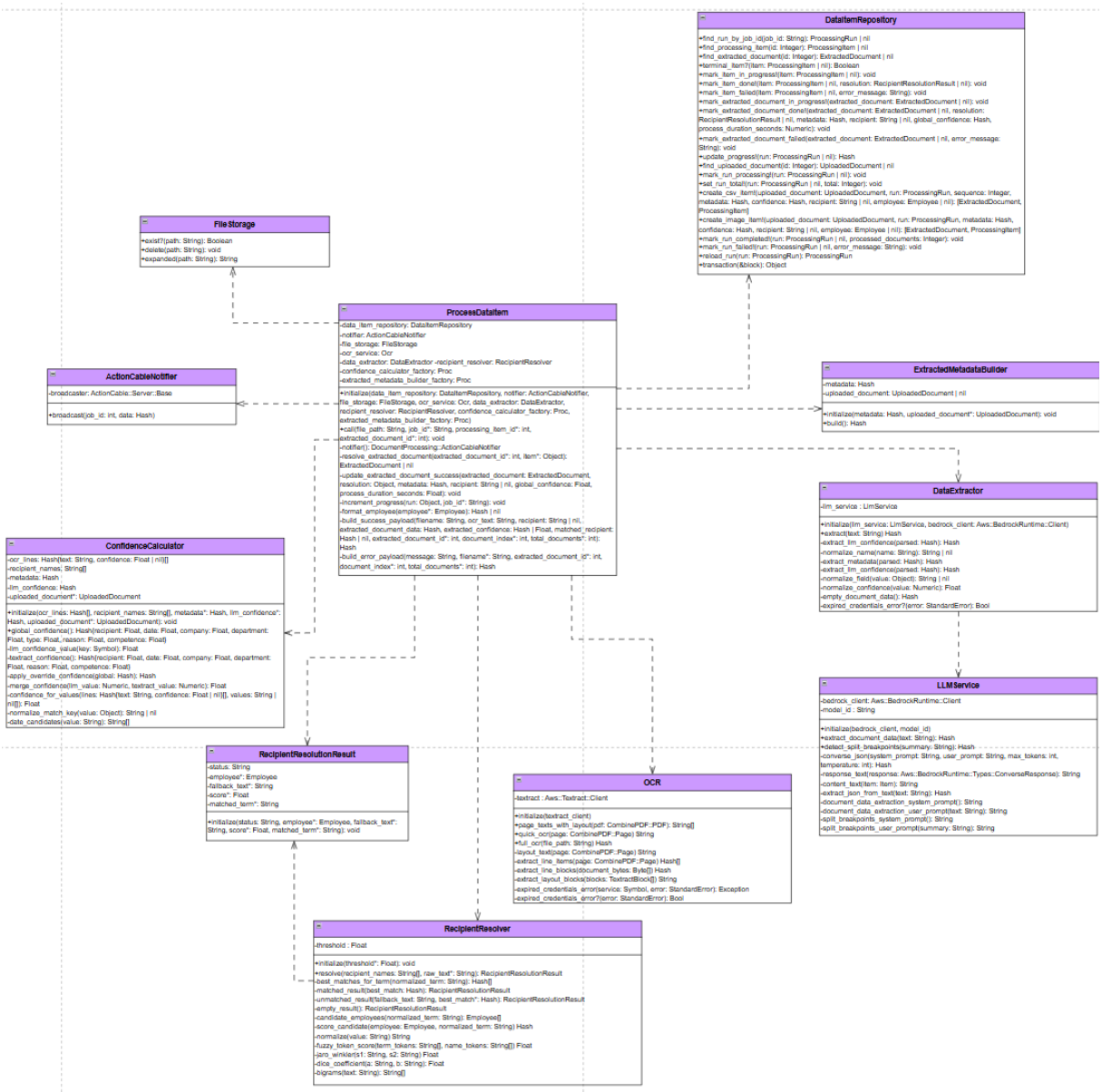


Figura 13: Diagramma classi parziale – Flusso di estrazione da PDF

Il diagramma evidenzia la catena di elaborazione dei documenti PDF estratti, con orchestrazione dei servizi OCR/LLM, persistenza dei metadati e aggiornamento dello stato dei processing item.

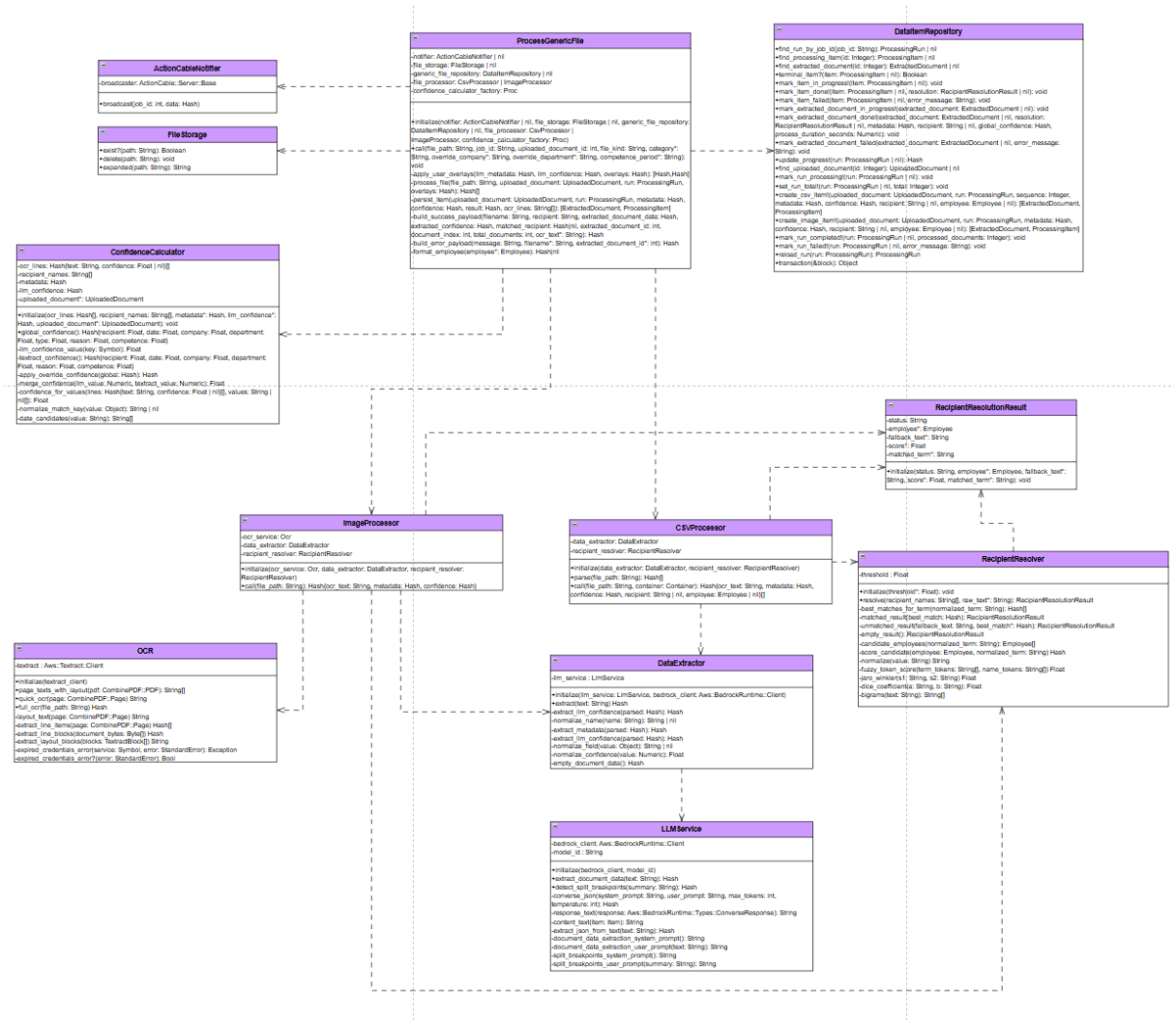


Figura 14: Diagramma classi parziale – Flusso file generici

Questo diagramma mostra il percorso dedicato ai file non PDF (ad esempio immagini o CSV), con inizializzazione specifica, job asincrono dedicato e normalizzazione del risultato.

5.2.2 Diagrammi di sequenza del blocco OCR

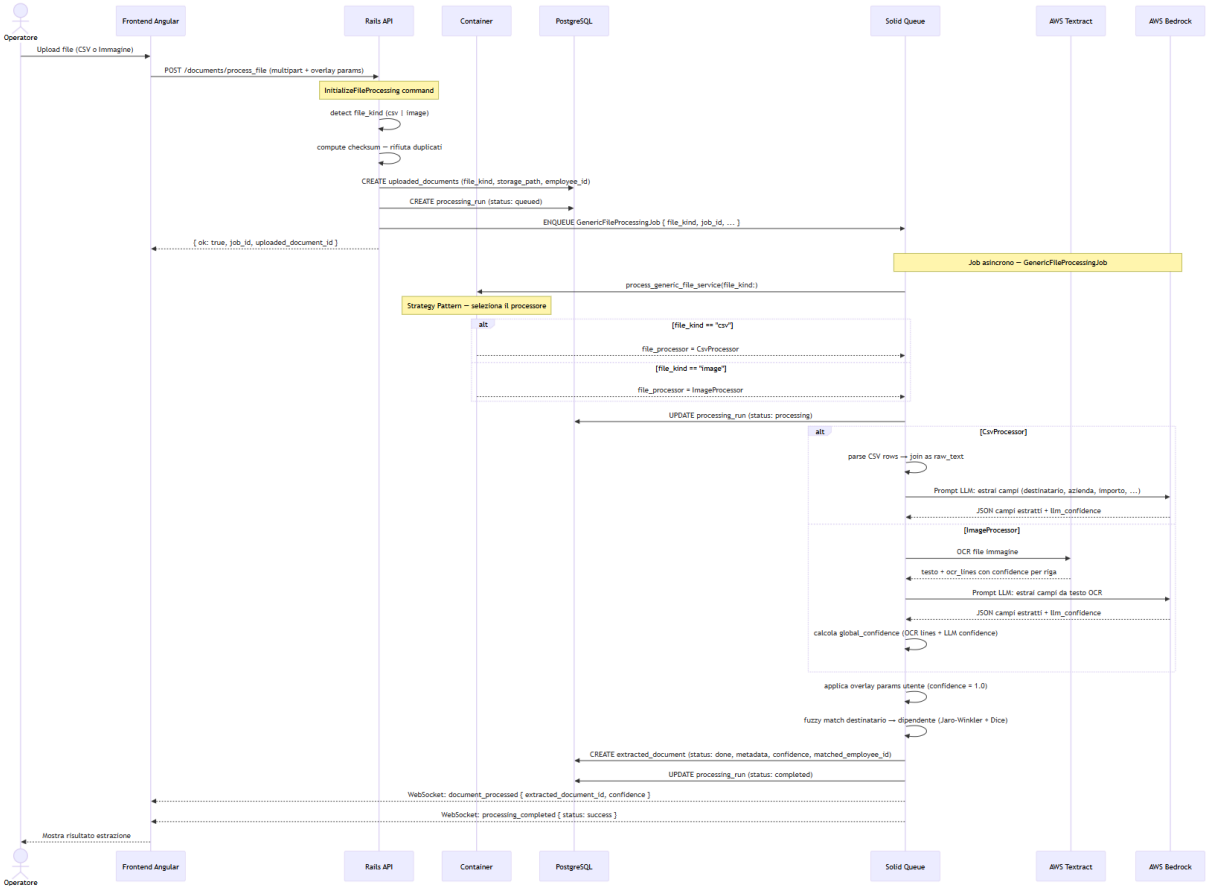


Figura 15: Diagramma di sequenza – Flusso Generic File

Il diagramma descrive la sequenza operativa per i file generici, attualmente immagini e csv: richiesta al controller, inizializzazione del processing dedicato, accodamento job e aggiornamento asincrono dello stato.

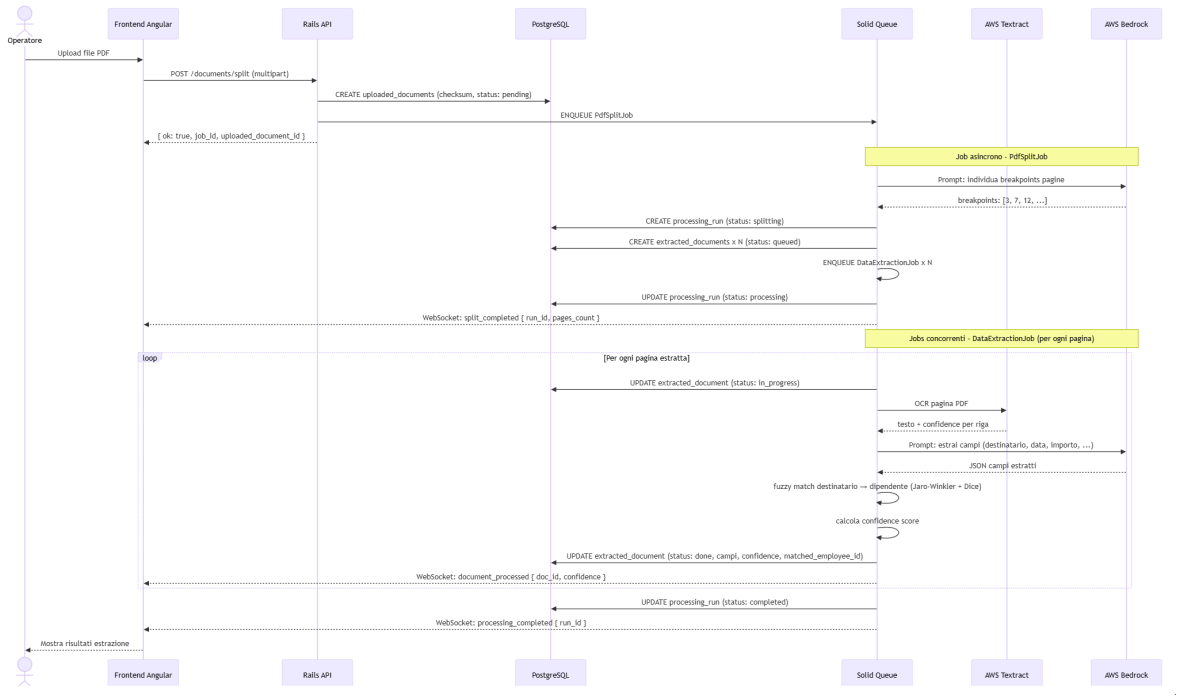


Figura 16: Diagramma di sequenza – Flusso PDF/OCR

Il diagramma mostra il flusso PDF con OCR: upload e split, orchestrazione dei servizi di estrazione, invocazione dei job asincroni e persistenza progressiva dei risultati estratti. Il flusso è separato dal percorso dei file generici, evidenziando la specificità del processo di splitting e dell'estrazione basata su pagine.

5.2.2.1 AIDocumentsController

AIDocumentsController

+split(pdf: UploadedFile, category*: String, company*: String, department*: String, competence_period*: String): void
+extracted_index(uploaded_document_id: int): void
+uploads(): void
+uploaded_file(id: int): void
+extracted_show(id: int): void
+extracted_pdf(id: int): void
+reassign_range(id: int, page_range*: String, page_start*: int, page_end*: int): void
+update_metadata(id: int, metadata_updates: Hash): void
+validate_extracted(id: int): Json
+retry_processing(): Json
+retry_extracted(): Json
+process_file(file: UploadedFile, category*: String, company*: String, department*: String, competence_period*: String): void
-parse_range_params(): [int ,int]
-int_or_nil(value: Object): int | nil
-render_error(message: String): void
-page_range_pdf_service(source_pdf_path: String): DocumentProcessing::PageRangePdf
-initialize_processing_command(): Object
-initialize_file_processing_command(): Object
-reassign_extracted_range_command(): Object
-extracted_document_presenter(document: ExtractedDocument): Object
-db_manager(): DbManager
-document_processing_container(): Container
-content_type_for_uploaded(uploaded_document): String

Classe di tipo Controller che espone gli endpoint per la gestione del ciclo di vita dei documenti nel modulo AI CoPilot. Coordina le richieste HTTP provenienti dal client per operazioni quali il caricamento dei file, l'avvio del processo di splitting, la validazione dei dati estratti e la riassegnazione manuale dei range di pagine.

Attributi

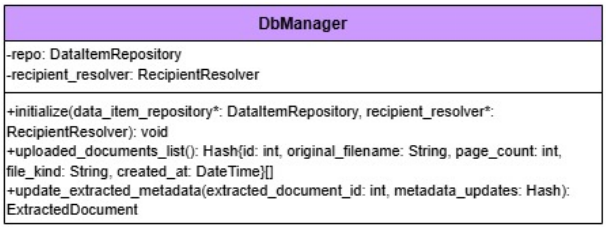
- Nessuno

Metodi

- split(pdf: UploadedFile, category*: String, company*: String, department*: String, competence_period*: String): void
- test_data(pdf: UploadedFile): void
- extracted_index(uploaded_document_id: int): void
- uploads(): void
- uploaded_file(id: int): void
- extracted_show(id: int): void
- extracted_pdf(id: int): void

- reassign_range(id: int, page_range*: String, page_start*: int, page_end*: int): void
- update_metadata(id: int, metadata_updates: Hash): void
- validate_extracted(id: int): Json
- retry_processing(): Json
- retry_extracted(): Json
- process_file(file: UploadedFile, category*: String, company*: String, department*: String, competence_period*: String): void
- parse_range_params(): [int ,int]
- int_or_nil(value: Object): int | nil
- render_error(message: String): void
- page_range_pdf_service(source_pdf_path: String): DocumentProcessing::PageRangePdf
- initialize_processing_command(): Object
- initialize_file_processing_command(): Object
- enqueue_single_data_extraction_command(): Object
- reassign_extracted_range_command(): Object
- extracted_document_presenter(document: ExtractedDocument): Object
- db_manager(): DbManager
- document_processing_container(): Container
- content_type_for_uploaded(uploaded_document): String

5.2.2.2 DbManager



Componente delegato all'interazione con il database per il modulo di elaborazione documentale. Agisce come un livello di astrazione per il recupero e l'aggiornamento dei metadati dei documenti estratti, incapsulando l'uso dei repository e dei resolver per l'associazione dei destinatari.

Attributi

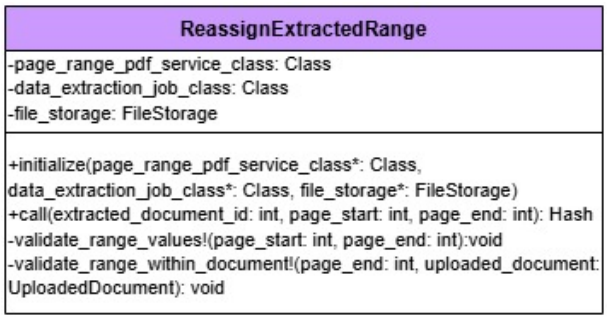
- repo: DataItemRepository
- recipient_resolver: RecipientResolver

Metodi

- initialize(data_item_repository*: DataItemRepository, recipient_resolver*: RecipientResolver): void

- uploaded_documents_list(): Hashid: int, original_filename: String, page_count: int, file_-kind: String, created_at: DateTime[]
- update_extracted_metadata(extracted_document_id: int, metadata_updates: Hash): ExtractedDocument

5.2.2.3 ReassignExtractedRange



Servizio dedicato alla modifica del range di pagine associato a un documento già estratto. In caso di errore nel riconoscimento automatico, permette di ricalcolare i metadati eseguendo un nuovo job di estrazione dati basato sulle nuove pagine selezionate dall'utente.

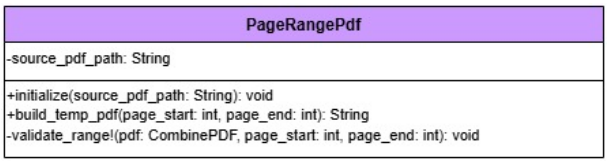
Attributi

- page_range_pdf_service_class: Class
- data_extraction_job_class: Class
- file_storage: FileStorage

Metodi

- initialize(page_range_pdf_service_class*: Class, data_extraction_job_class*: Class, file_-storage*: FileStorage)
- call(extracted_document_id: int, page_start: int, page_end: int): Hash
- validate_range_values!(page_start: int, page_end: int):void
- validate_range_within_document!(page_end: int, uploaded_document: UploadedDocument): void

5.2.2.4 PageRangePdf



Servizio di utilità che si occupa della manipolazione fisica dei file PDF. È responsabile della creazione di file temporanei estrapolando un sottoinsieme specifico di pagine a partire da un documento sorgente più ampio.

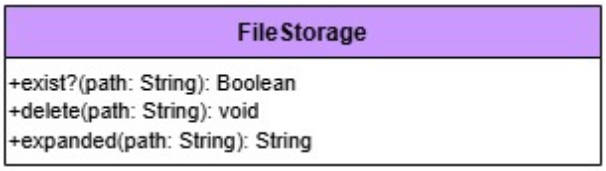
Attributi

- source_pdf_path: String

Metodi

- initialize(source_pdf_path: String): void
- build_temp_pdf(page_start: int, page_end: int): String
- validate_range!(pdf: CombinePDF, page_start: int, page_end: int): void

5.2.2.5 FileStorage



Interfaccia di astrazione per la gestione del file system. Offre metodi standardizzati per verificare l'esistenza, eliminare ed espandere i percorsi dei file temporanei e persistenti utilizzati durante le fasi di caricamento e analisi.

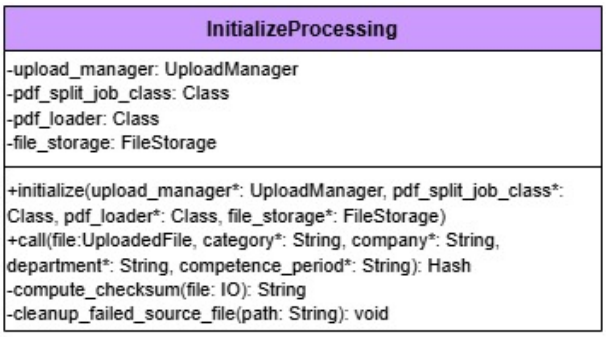
Attributi

- Nessuno

Metodi

- exist?(path: String): Boolean
- delete(path: String): void
- expanded(path: String): String

5.2.2.6 InitializeProcessing



Componente di orchestrazione che gestisce le fasi preparatorie all'elaborazione di un file,. Verifica l'integrità del caricamento, calcola i checksum e innesca la catena di elaborazione asincrona (come lo splitting del PDF) tramite job dedicati.

Attributi

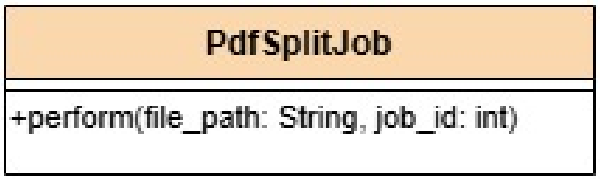
- upload_manager: UploadManager

- pdf_split_job_class: Class
- pdf_loader: Class
- file_storage: FileStorage

Metodi

- initialize(upload_manager*: UploadManager, pdf_split_job_class*: Class, pdf_loader*: Class, file_storage*: FileStorage)
- call(file:UploadedFile, category*: String, company*: String, department*: String, competence_period*: String): Hash
- compute_checksum(file: IO): String
- cleanup_failed_source_file(path: String): void

5.2.2.7 PdfSplitJob



Job in background responsabile dell'esecuzione asincrona del processo di divisione dei file PDF. Viene richiamato per alleggerire il carico sul thread principale, delegando la separazione logica dei documenti massivi in file individuali.

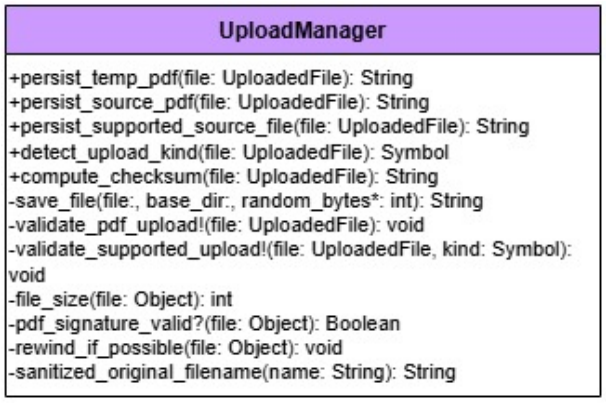
Attributi

- Nessuno

Metodi

- perform(file_path: String, job_id: int): void

5.2.2.8 UploadManager



Servizio centralizzato per la validazione e la persistenza dei file in ingresso. Gestisce il salvataggio dei file sorgente, verifica la correttezza del formato (es. firme PDF) e ne determina la tipologia per indirizzarli al corretto flusso di elaborazione.

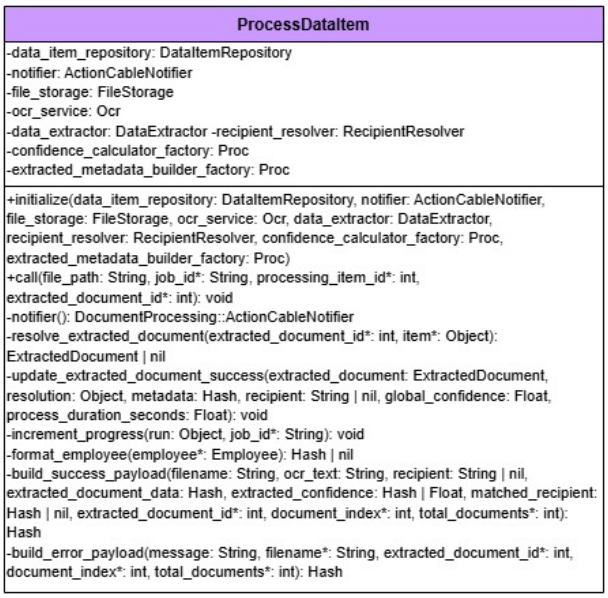
Attributi

- Nessuno

Metodi

- persist_temp_pdf(file: UploadedFile): String
- persist_source_pdf(file: UploadedFile): String
- persist_supported_source_file(file: UploadedFile): String
- detect_upload_kind(file: UploadedFile): Symbol
- compute_checksum(file: UploadedFile): String
- save_file(file: UploadedFile, base_dir: String, random_bytes: int): String
- validate_pdf_upload!(file: UploadedFile): void
- validate_supported_upload!(file: UploadedFile, kind: Symbol): void
- file_size(file: Object): int
- pdf_signature_valid?(file: Object): Boolean
- rewind_if_possible(file: Object): void
- sanitized_original_filename(name: String): String

5.2.2.9 EnqueueSingleDataExtraction



Comando delegato all'inserimento in coda di un job di estrazione dati per un singolo documento. Fa da ponte tra l'avvenuto caricamento del file e l'inizio dell'elaborazione tramite intelligenza artificiale.

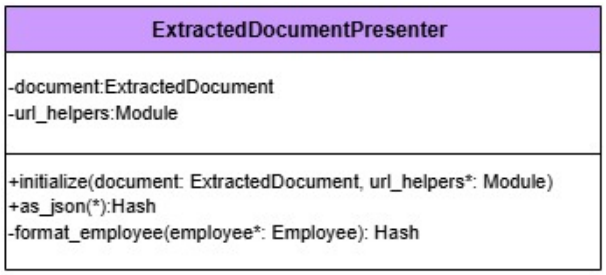
Attributi

- upload_manager: UploadManager
- data_extraction_job_class: Class

Metodi

- initialize(upload_manager*: Class, data_extraction_job_class*: Class)
- call(file: UploadedFile): Hash

5.2.2.10 ExtractedDocumentPresenter



Classe di presentazione (Serializer) responsabile della formattazione dei dati estratti. Trasforma le entità complesse, inclusi i dati anagrafici dei dipendenti associati, in una struttura JSON ottimizzata per la renderizzazione lato frontend.

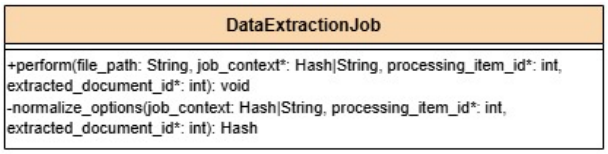
Attributi

- document:ExtractedDocument
- url_helpers:Module

Metodi

- initialize(document: ExtractedDocument, url_helpers*: Module)
- as_json(*):Hash
- format_employee(employee*: Employee): Hash

5.2.2.11 DataExtractionJob



Job in background che esegue materialmente il processo di estrazione dati su un singolo documento o frammento di esso. Normalizza il contesto di esecuzione e coordina le chiamate ai servizi AI in modo asincrono.

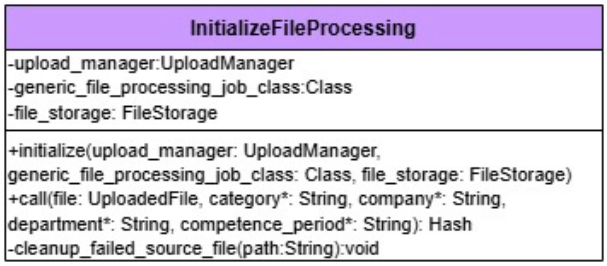
Attributi

- Nessuno

Metodi

- perform(file_path: String, job_context*: Hash|String, processing_item_id*: int, extracted_document_id*: int): void
- normalize_options(job_context: Hash|String, processing_item_id*: int, extracted_document_id*: int): Hash

5.2.2.12 InitializeFileProcessing



Servizio analogo a *InitializeProcessing*, ma specializzato per la gestione iniziale di file generici che non richiedono processi di splitting complessi (come le immagini o i CSV).

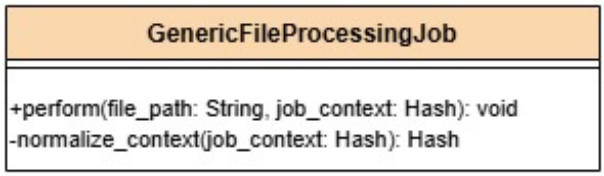
Attributi

- upload_manager: UploadManager
- generic_file_processing_job_class: Class
- file_storage: FileStorage

Metodi

- initialize(upload_manager: UploadManager, generic_file_processing_job_class: Class, file_storage: FileStorage)
- call(file: UploadedFile, category*: String, company*: String, department*: String, competence_period*: String): Hash
- cleanup_failed_source_file(path:String):void

5.2.2.13 GenericFileProcessingJob



Job in background incaricato di processare file di formati eterogenei (es. file non PDF) in maniera asincrona, applicando la specifica logica di estrazione in base al tipo di file.

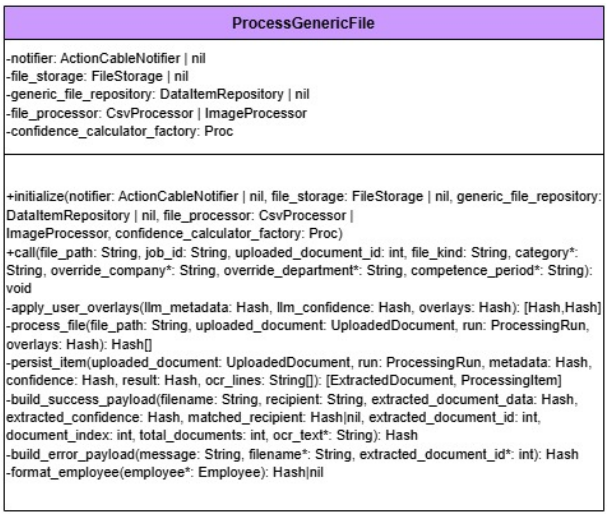
Attributi

- Nessuno

Metodi

- perform(file_path: String, job_context: Hash): void
- normalize_context(job_context: Hash): Hash

5.2.2.14 ProcessGenericFile



Servizio core per l'elaborazione dei file generici (immagini, CSV). Smista il documento al processore corretto, integra i punteggi di confidenza restituiti dall'LLM e prepara i payload (sia in caso di successo che di errore) per l'aggiornamento in tempo reale dell'interfaccia.

Attributi

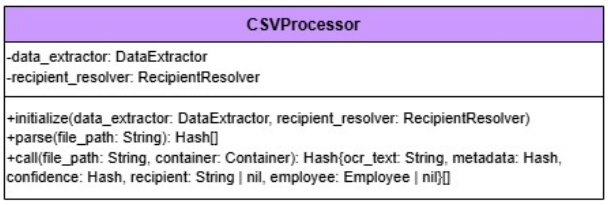
- notifier: ActionCableNotifier | nil
- file_storage: FileStorage | nil
- generic_file_repository: DataItemRepository | nil
- image_processor_factory: Proc
- csv_processor_factory: Proc

Metodi

- initialize(notifier: ActionCableNotifier | nil, file_storage: FileStorage | nil, generic_file_repository: DataItemRepository | nil, file_processor: CsvProcessor | ImageProcessor, confidence_calculator_factory: Proc)
- call(file_path: String, job_id: String, uploaded_document_id: int, file_kind: String, category*: String, override_company*: String, override_department*: String, competence_period*: String): void
- apply_user_overlays(llm_metadata: Hash, llm_confidence: Hash, overlays: Hash): [Hash,Hash]
- process_file(file_path: String, uploaded_document: UploadedDocument, run: ProcessingRun, overlays: Hash): Hash[]
- persist_item(uploaded_document: UploadedDocument, run: ProcessingRun, metadata: Hash, confidence: Hash, result: Hash, ocr_lines: String[]): [ExtractedDocument, ProcessingItem]

- build_success_payload(filename: String, recipient: String, extracted_document_data: Hash, extracted_confidence: Hash, matched_recipient: Hash|nil, extracted_document_id: int, document_index: int, total_documents: int, ocr_text*: String): Hash
- build_error_payload(message: String, filename*: String, extracted_document_id*: int): Hash
- format_employee(employee*: Employee): Hash|nil

5.2.2.15 CSVProcessor



Servizio specializzato nell'estrazione di dati da file CSV. Implementa logiche di parsing e normalizzazione specifiche per questo formato, estraendo i metadati strutturati e preparando i dati per la valutazione della confidence da parte dell'LLM.

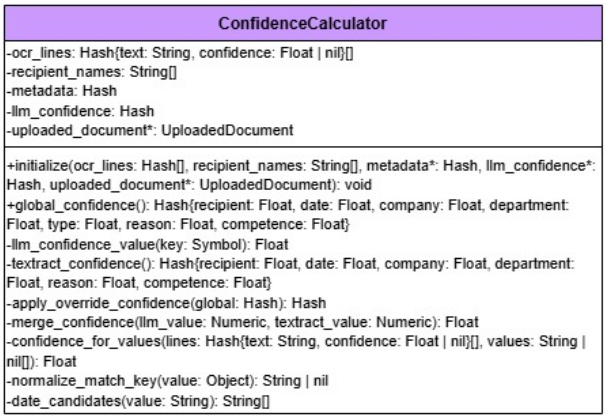
Attributi

- data_extractor: DataExtractor
- recipient_resolver: RecipientResolver

Metodi

- initialize(data_extractor: DataExtractor, recipient_resolver: RecipientResolver)
- parse(file_path: String): Hash[]
- call(file_path: String, container: Container): Hashocr_text: String, metadata: Hash, confidence: Hash, recipient: String | nil, employee: Employee | nil[]

5.2.2.16 ConfidenceCalculator



Motore di calcolo dedicato alla determinazione del punteggio di affidabilità (confidence score) dei dati estratti[cite: 817, 826]. Combina le metriche restituite dal motore OCR con quelle dell'LLM, applicando logiche di fusione e regole di override in base a campi specifici (es. destinatari, date).

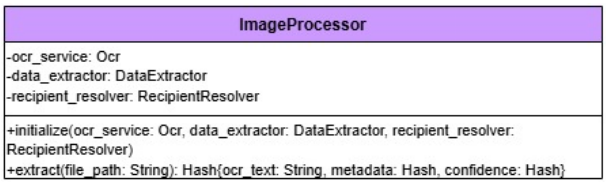
Attributi

- ocr_lines: Hashtext: String, confidence: Float | nil[]
- recipient_names: String[]
- metadata: Hash
- llm_confidence: Hash
- uploaded_document*: UploadedDocument

Metodi

- initialize(ocr_lines: Hash[], recipient_names: String[], metadata*: Hash, llm_confidence*: Hash, uploaded_document*: UploadedDocument): void
- global_confidence(): Hashrecipient: Float, date: Float, company: Float, department: Float, type: Float, reason: Float, competence: Float
- llm_confidence_value(key: Symbol): Float
- textract_confidence(): Hashrecipient: Float, date: Float, company: Float, department: Float, reason: Float, competence: Float
- apply_override_confidence(global: Hash): Hash
- merge_confidence(llm_value: Numeric, textract_value: Numeric): Float
- confidence_for_values(lines: Hashtext: String, confidence: Float | nil[], values: String | nil[]): Float
- normalize_match_key(value: Object): String | nil
- date_candidates(value: String): String[]

5.2.2.17 ImageProcessor



Servizio che coordina il flusso di estrazione dei dati a partire da formati immagine. Incapsula le chiamate al servizio OCR per ottenere il testo grezzo e lo inoltra al modulo di estrazione per identificare i metadati strutturati e i destinatari.

Attributi

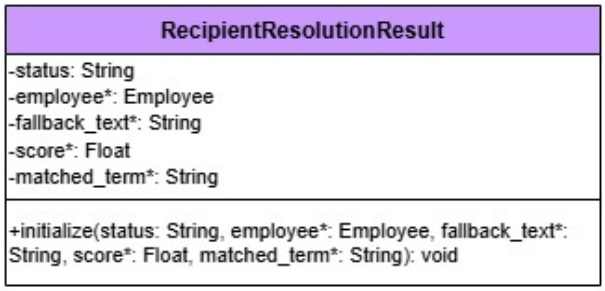
- ocr_service: Ocr
- data_extractor: DataExtractor

- recipient_resolver: RecipientResolver

Metodi

- initialize(ocr_service: Ocr, data_extractor: DataExtractor, recipient_resolver: RecipientResolver)
- extract(file_path: String): Hashocr_text: String, metadata: Hash, confidence: Hash

5.2.2.18 RecipientResolutionResult



Data Transfer Object (DTO) che modella il risultato del processo di identificazione di un destinatario[cite: 846, 854]. Contiene lo stato della ricerca, il dipendente associato, eventuali testi di fallback e il punteggio di accuratezza del match.

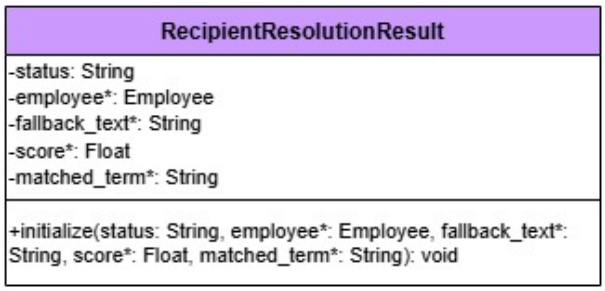
Attributi

- status: String
- employee*: Employee
- fallback_text*: String
- score*: Float
- matched_term*: String

Metodi

- initialize(status: String, employee*: Employee, fallback_text*: String, score*: Float, matched_term*: String): void

5.2.2.19 RecipientResolver



Motore di risoluzione anagrafica che associa i nomi estratti dal testo ai record reali dei dipendenti[cite: 855, 860]. Impiega algoritmi di string matching avanzati (come Jaro-Winkler e coefficiente di Dice) per valutare i candidati e restituire l'associazione più probabile.

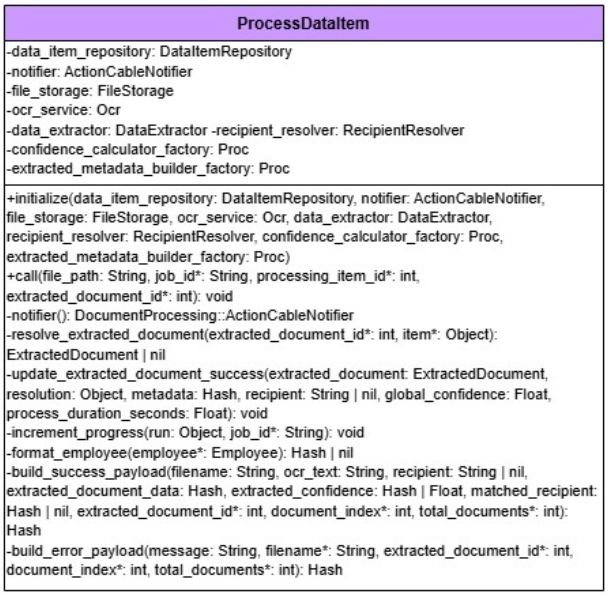
Attributi

- threshold : Float

Metodi

- initialize(threshold*: Float): void
- resolve(recipient_names: String[], raw_text*: String): RecipientResolutionResult
- best_matches_for_term(normalized_term: String): Hash[]
- matched_result(best_match: Hash): RecipientResolutionResult
- unmatched_result(fallback_text: String, best_match*: Hash): RecipientResolutionResult
- empty_result(): RecipientResolutionResult
- candidate_employees(normalized_term: String): Employee[]
- score_candidate(employee: Employee, normalized_term: String): Hash
- normalize(value: String): String
- fuzzy_token_score(term_tokens: String[], name_tokens: String[]): Float
- jaro_winkler(s1: String, s2: String): Float
- dice_coefficient(a: String, b: String): Float
- bigrams(text: String): String[]

5.2.2.20 ProcessDataItem



Servizio orchestratore per l'elaborazione end-to-end di un singolo elemento di dato. Coordina l'interazione tra OCR, Data Extractor, Recipient Resolver e gestisce l'aggiornamento dello stato del job, inviando notifiche WebSockets (ActionCable) al termine del processo.

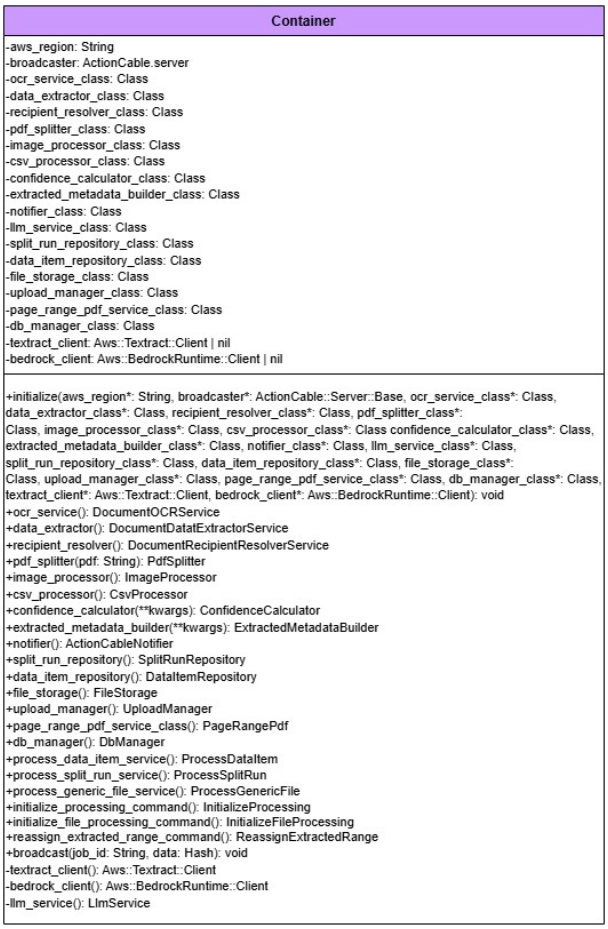
Attributi

- data_item_repository: DataItemRepository
- notifier: ActionCableNotifier
- file_storage: FileStorage
- ocr_service: Ocr
- data_extractor: DataExtractor
- recipient_resolver: RecipientResolver
- confidence_calculator_factory: Proc
- extracted_metadata_builder_factory: Proc

Metodi

- initialize(data_item_repository: DataItemRepository, notifier: ActionCableNotifier, file_storage: FileStorage, ocr_service: Ocr, data_extractor: DataExtractor, recipient_resolver: RecipientResolver, confidence_calculator_factory: Proc, extracted_metadata_builder_factory: Proc)
- call(file_path: String, job_id*: String, processing_item_id*: int, extracted_document_id*: int): void
- notifier(): DocumentProcessing::ActionCableNotifier
- resolve_extracted_document(extracted_document_id*: int, item*: Object): ExtractedDocument | nil
- update_extracted_document_success(extracted_document: ExtractedDocument, resolution: Object, metadata: Hash, recipient: String | nil, global_confidence: Float, process_duration_seconds: Float): void
- increment_progress(run: Object, job_id*: String): void
- format_employee(employee*: Employee): Hash | nil
- build_success_payload(filename: String, ocr_text: String, recipient: String | nil, extracted_document_data: Hash, extracted_confidence: Hash | Float, matched_recipient: Hash | nil, extracted_document_id*: int, document_index*: int, total_documents*: int): Hash
- build_error_payload(message: String, filename*: String, extracted_document_id*: int, document_index*: int, total_documents*: int): Hash

5.2.2.21 Container



Modulo di Dependency Injection (DI) del sistema. Centralizza la configurazione e l'istanziazione di tutte le classi di servizio, repository e client esterni (es. AWS Bedrock, AWS Textract) garantendo una corretta inversione delle dipendenze.

Attributi

- aws_region: String
- broadcaster: ActionCable.server
- ocr_service_class: Class
- data_extractor_class: Class
- recipient_resolver_class: Class
- pdf_splitter_class: Class
- image_processor_class: Class
- csv_processor_class: Class
- confidence_calculator_class: Class
- extracted_metadata_builder_class: Class
- notifier_class: Class

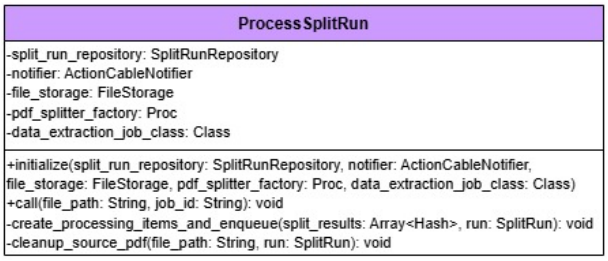
- llm_service_class: Class
- split_run_repository_class: Class
- data_item_repository_class: Class
- file_storage_class: Class
- upload_manager_class: Class
- page_range_pdf_service_class: Class
- db_manager_class: Class
- textract_client: Aws::Textract::Client | nil
- bedrock_client: Aws::BedrockRuntime::Client | nil

Metodi

- initialize(aws_region*: String, broadcaster*: ActionCable::Server::Base, ocr_service_class*: Class, data_extractor_class*: Class, recipient_resolver_class*: Class, pdf_splitter_class*: Class, image_processor_class*: Class, csv_processor_class*: Class confidence_calculator_class*: Class, extracted_metadata_builder_class*: Class, notifier_class*: Class, llm_service_class*: Class, split_run_repository_class*: Class, data_item_repository_class*: Class, file_storage_class*: Class, upload_manager_class*: Class, page_range_pdf_service_class*: Class, db_manager_class*: Class, textract_client*: Aws::Textract::Client, bedrock_client*: Aws::BedrockRuntime::Client): void
- ocr_service(): DocumentOCRService
- data_extractor(): DocumentDataExtractorService
- recipient_resolver(): DocumentRecipientResolverService
- pdf_splitter(pdf: String): PdfSplitter
- image_processor(): ImageProcessor
- csv_processor(): CsvProcessor
- confidence_calculator(**kwargs): ConfidenceCalculator
- extracted_metadata_builder(**kwargs): ExtractedMetadataBuilder
- notifier(): ActionCableNotifier
- split_run_repository(): SplitRunRepository
- data_item_repository(): DataItemRepository
- file_storage(): FileStorage
- upload_manager(): UploadManager
- page_range_pdf_service_class(): PageRangePdf
- db_manager(): DbManager
- process_data_item_service(): ProcessDataItem
- process_split_run_service(): ProcessSplitRun
- process_generic_file_service(): ProcessGenericFile

- initialize_processing_command(): InitializeProcessing
- initialize_file_processing_command(): InitializeFileProcessing
- enqueue_single_data_extraction_command(): EnqueueSingleDataExtraction
- reassign_extracted_range_command(): ReassignExtractedRange
- broadcast(job_id: String, data: Hash): void
- textract_client(): Aws::Textract::Client
- bedrock_client(): Aws::BedrockRuntime::Client
- llm_service(): LlmService

5.2.2.22 ProcessSplitRun



Servizio che orchestra il ciclo di vita post-split di un documento PDF. Recupera i segmenti separati, crea i relativi item di elaborazione nel database e ne avvia la messa in coda per l'estrazione asincrona dei dati.

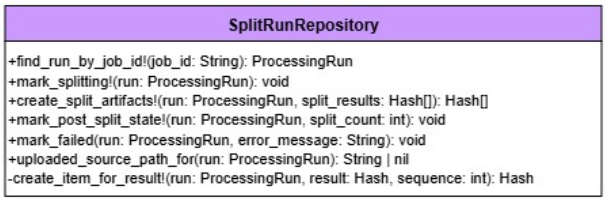
Attributi

- split_run_repository: SplitRunRepository
- notifier: ActionCableNotifier
- file_storage: FileStorage
- pdf_splitter_factory: Proc
- data_extraction_job_class: Class

Metodi

- initialize(split_run_repository: SplitRunRepository, notifier: ActionCableNotifier, file_storage: FileStorage, pdf_splitter_factory: Proc, data_extraction_job_class: Class)
- call(file_path: String, job_id: String): void
- create_processing_items_and_enqueue(split_results: Array<Hash>, run: SplitRun): void
- cleanup_source_pdf(file_path: String, run: SplitRun): void

5.2.2.23 SplitRunRepository



Classe di tipo Repository dedicata alla gestione dello stato per le esecuzioni di splitting. Fornisce l'interfaccia di persistenza per tracciare i progressi dell'elaborazione massiva, segnandone successi, artefatti creati o eventuali fallimenti.

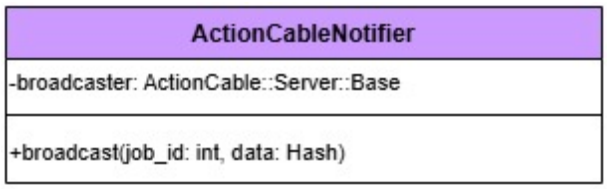
Attributi

- Nessuno

Metodi

- find_run_by_job_id!(job_id: String): ProcessingRun
- mark_splitting!(run: ProcessingRun): void
- create_split_artifacts!(run: ProcessingRun, split_results: Hash[]): Hash[]
- mark_post_split_state!(run: ProcessingRun, split_count: int): void
- mark_failed(run: ProcessingRun, error_message: String): void
- uploaded_source_path_for(run: ProcessingRun): String | nil
- create_item_for_result!(run: ProcessingRun, result: Hash, sequence: int): Hash

5.2.2.24 ActionCableNotifier



Servizio wrapper per la gestione delle notifiche in tempo reale. Permette al backend di comunicare al frontend via WebSockets lo stato di avanzamento dei processi asincroni (job) per un aggiornamento fluido della UI.

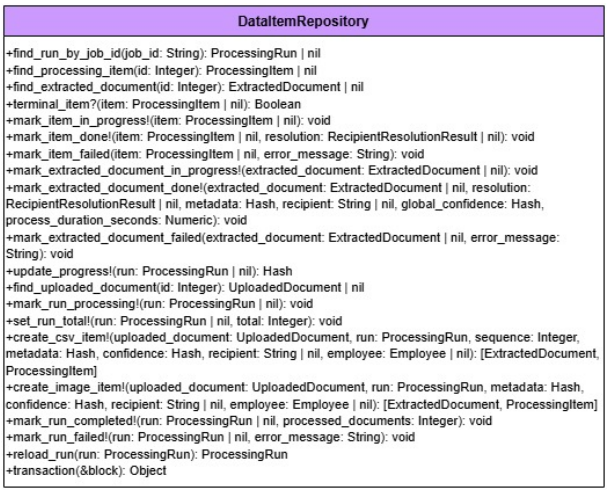
Attributi

- broadcaster: ActionCable::Server::Base

Metodi

- broadcast(job_id: int, data: Hash)

5.2.2.25 DataItemRepository



Repository che gestisce la persistenza degli oggetti legati alla singola elaborazione e all'estrazione documentale. Regola le transazioni sul database e la transizione di stato degli item da "in progress" a "completato" o "fallito".

Attributi

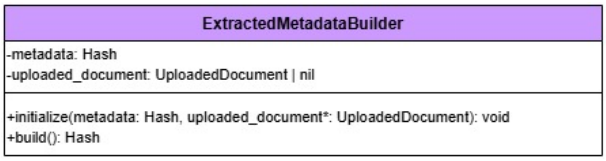
- Nessuno

Metodi

- find_run_by_job_id(job_id: String): ProcessingRun | nil
- find_processing_item(id: Integer): ProcessingItem | nil
- find_extracted_document(id: Integer): ExtractedDocument | nil
- terminal_item?(item: ProcessingItem | nil): Boolean
- mark_item_in_progress!(item: ProcessingItem | nil): void
- mark_item_done!(item: ProcessingItem | nil, resolution: RecipientResolutionResult | nil): void
- mark_item_failed(item: ProcessingItem | nil, error_message: String): void
- mark_extracted_document_in_progress!(extracted_document: ExtractedDocument | nil): void
- mark_extracted_document_done!(extracted_document: ExtractedDocument | nil, resolution: RecipientResolutionResult | nil, metadata: Hash, recipient: String | nil, global_confidence: Hash, process_duration_seconds: Numeric): void
- mark_extracted_document_failed(extracted_document: ExtractedDocument | nil, error_message: String): void
- update_progress!(run: ProcessingRun | nil): Hash
- find_uploaded_document(id: Integer): UploadedDocument | nil
- mark_run_processing!(run: ProcessingRun | nil): void

- set_run_total!(run: ProcessingRun | nil, total: Integer): void
- create_csv_item!(uploaded_document: UploadedDocument, run: ProcessingRun, sequence: Integer, metadata: Hash, confidence: Hash, recipient: String | nil, employee: Employee | nil): [ExtractedDocument, ProcessingItem]
- create_image_item!(uploaded_document: UploadedDocument, run: ProcessingRun, metadata: Hash, confidence: Hash, recipient: String | nil, employee: Employee | nil): [ExtractedDocument, ProcessingItem]
- mark_run_completed!(run: ProcessingRun | nil, processed_documents: Integer): void
- mark_run_failed!(run: ProcessingRun | nil, error_message: String): void
- reload_run(run: ProcessingRun): ProcessingRun
- transaction(&block): Object

5.2.2.26 ExtractedMetadataBuilder



Classe di utilità responsabile della costruzione e formattazione finale dell'oggetto dei metadati estratti, preparandolo per essere associato in maniera coerente al modello del database.

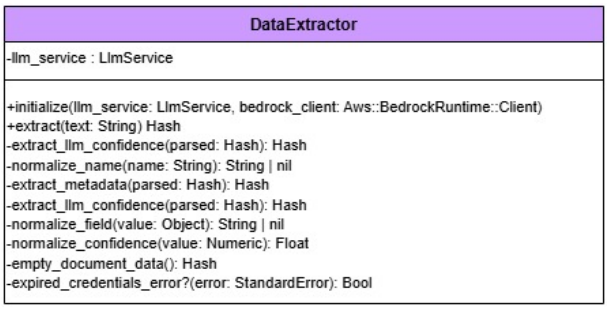
Attributi

- metadata: Hash
- uploaded_document: UploadedDocument | nil

Metodi

- initialize(metadata: Hash, uploaded_document: UploadedDocument): void
- build(): Hash

5.2.2.27 DataExtractor



Servizio delegato all'interfacciamento con il modello LLM per estrapolare metadati strutturati dal testo grezzo[cite: 1022, 1030]. Include logiche per la pulizia del risultato JSON e la normalizzazione dei campi estratti prima del salvataggio.

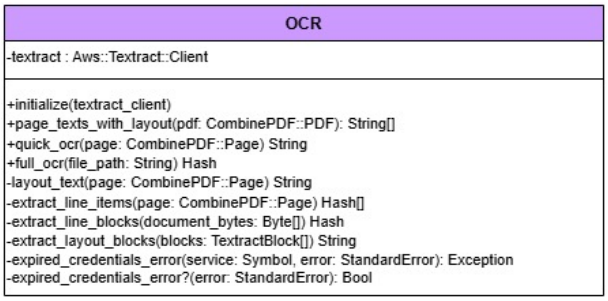
Attributi

- llm_service : LlmService

Metodi

- initialize(llm_service: LlmService, bedrock_client: Aws::BedrockRuntime::Client)
- extract(text: String): Hash
- extract_llm_confidence(parsed: Hash): Hash
- normalize_name(name: String): String | nil
- extract_metadata(parsed: Hash): Hash
- extract_llm_confidence(parsed: Hash): Hash
- normalize_field(value: Object): String | nil
- normalize_confidence(value: Numeric): Float
- empty_document_data(): Hash
- expired_credentials_error?(error: StandardError): Bool

5.2.2.28 OCR



Servizio che wrappa le chiamate al motore OCR (AWS Textract). È responsabile della conversione di PDF o immagini in blocchi di testo strutturati, analizzando i layout per fornire un input coerente all'intelligenza artificiale generativa.

Attributi

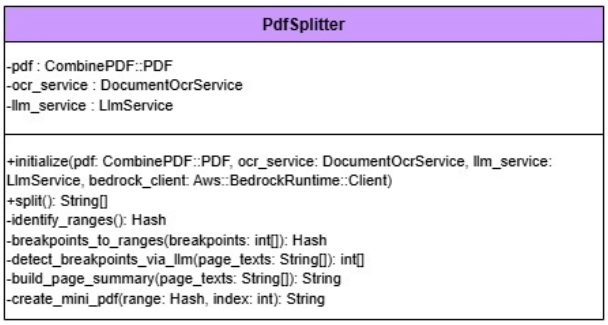
- textract : Aws::Textract::Client

Metodi

- initialize(textract_client)
- page_texts_with_layout(pdf: CombinePDF::PDF): String[]
- quick_ocr(page: CombinePDF::Page) String
- full_ocr(file_path: String) Hash
- layout_text(page: CombinePDF::Page) String
- extract_line_items(page: CombinePDF::Page) Hash[]

- extract_line_blocks(document_bytes: Byte[]) Hash
- extract_layout_blocks(blocks: TextractBlock[]) String
- expired_credentials_error(service: Symbol, error: StandardError): Exception
- expired_credentials_error?(error: StandardError): Bool

5.2.2.29 PdfSplitter



Motore di divisione dei PDF basato su intelligenza artificiale. Identifica i confini logici tra documenti diversi all'interno dello stesso file (breakpoints) basandosi su riassunti del testo di ogni pagina analizzati tramite l'LLM.

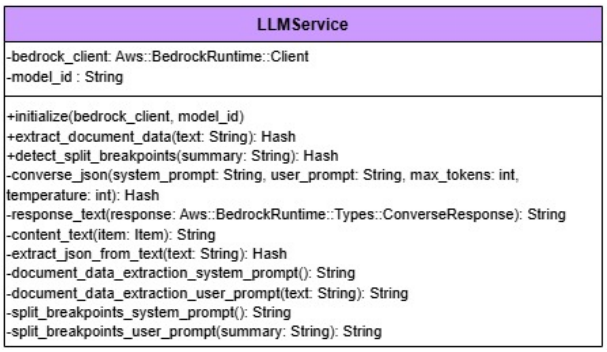
Attributi

- pdf : CombinePDF::PDF
- ocr_service : DocumentOcrService
- llm_service : LlmService

Metodi

- +initialize(pdf: CombinePDF::PDF, ocr_service: DocumentOcrService, llm_service: LlmService, bedrock_client: Aws::BedrockRuntime::Client)
- split(): String[]
- identify_ranges(): Hash
- breakpoints_to_ranges(breakpoints: int[]): Hash
- detect_breakpoints_via_llm(page_texts: String[]): int[]
- build_page_summary(page_texts: String[]): String
- create_mini_pdf(range: Hash, index: int): String

5.2.2.30 LLMService



Servizio dedicato alla gestione della comunicazione con i foundation models (es. tramite AWS Bedrock Runtime). Prepara i prompt di sistema e utente per l'estrazione dati e il rilevamento dei breakpoint, gestendo la corretta esecuzione delle richieste AI e il parsing delle risposte JSON.

Attributi

- bedrock_client: Aws::BedrockRuntime::Client
- model_id : String

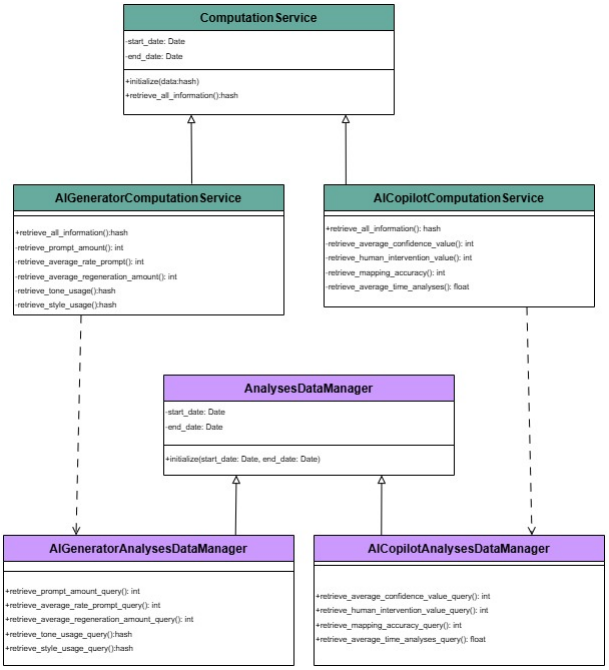
Metodi

- initialize(bedrock_client, model_id)
- extract_document_data(text: String): Hash
- detect_split_breakpoints(summary: String): Hash
- converse_json(system_prompt: String, user_prompt: String, max_tokens: int, temperature: int): Hash
- response_text(response: Aws::BedrockRuntime::Types::ConverseResponse): String
- content_text(item: Item): String
- extract_json_from_text(text: String): Hash
- document_data_extraction_system_prompt(): String
- document_data_extraction_user_prompt(text: String): String
- split_breakpoints_system_prompt(): String
- split_breakpoints_user_prompt(summary: String): String

5.3 Modulo Data analisi

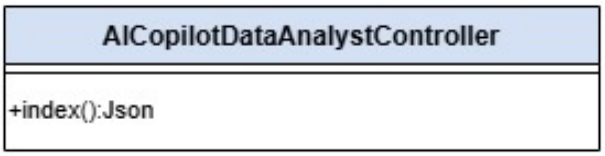
Il modulo *Data analisi* espone i componenti fondamentali per l'estrazione, il calcolo e l'esposizione delle metriche di utilizzo e delle statistiche relative alle due funzionalità principali del sistema: l'AI Co-Pilot per i documenti e l'AI-assistant generativo. Il modulo segue un'architettura a livelli, separando la gestione delle richieste (Controller), la logica di business e calcolo (Service) e l'interazione con il database (Data Manager).

5.3.1 Diagramma delle classi blocco modulo Data analisi



Questo diagramma illustra le classi principali del modulo di Data Analisi, evidenziando le loro responsabilità e le relazioni tra di esse. Lo scopo di questo modulo è fornire un'interfaccia coerente per l'estrazione e la presentazione delle metriche chiave, consentendo al front-end di visualizzare dashboard informative sull'utilizzo e le performance dell'AI Co-Pilot e dell'AI-assistant generativo.

5.3.1.1 AICopilotDataAnalystController



Classe di tipo Controller responsabile dell'esposizione degli endpoint API relativi alle statistiche dell'AI Co-Pilot. Gestisce le richieste HTTP provenienti dal front-end, validando i parametri di input (es. i filtri temporali). Delega la complessa logica di calcolo delle metriche (come confidenza media e accuratezza) al livello di servizio sottostante, per poi impacchettare i risultati in un formato standard (es. JSON) da restituire al client.

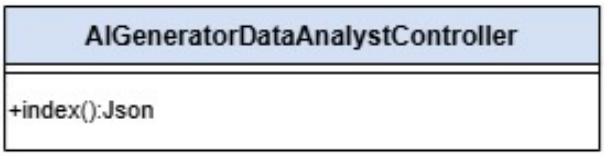
Attributi

- Nessuno

Metodi

- +index():Json

5.3.1.2 AIGeneratorDataAnalystController



Classe di tipo Controller che gestisce l'interfaccia API per le analytics dell'AI-assistant generativo. Riceve e processa le richieste relative ai dati di generazione (prompt effettuati, rigenerazioni, feedback degli utenti). Come il suo omologo per il Co-Pilot, funge da punto di ingresso per le richieste, interfacciandosi con i servizi di computazione per elaborare i dati e restituire le risposte al client in base ai parametri temporali specificati.

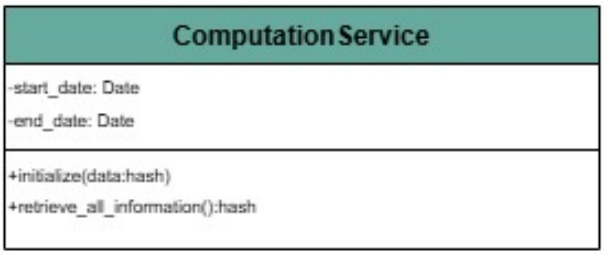
Attributi

- Nessuno

Metodi

- +index():Json

5.3.1.3 ComputationService



Interfaccia o classe base astratta che definisce il contratto standard per i servizi di calcolo delle metriche di analytics. Fornisce i metodi comuni e le firme necessarie per l'elaborazione dei dati grezzi, garantendo uniformità e polimorfismo tra i diversi servizi di computazione implementati nel modulo.

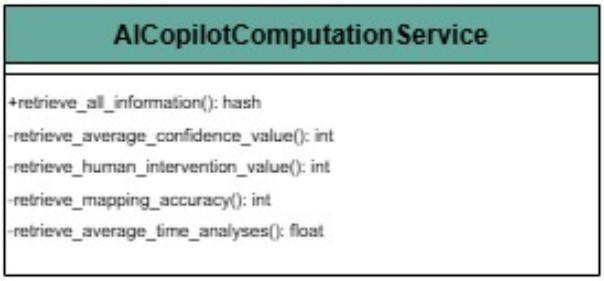
Attributi

- start_date: Date
- end_date: Date

Metodi

- initialize(data:hash)
- retrieve_all_information():hash

5.3.1.4 AiCopilotComputationService



Classe concreta che implementa la logica di business per l'elaborazione delle metriche specifiche dell'AI Co-Pilot. Riceve i dati grezzi dal livello dati e applica gli algoritmi necessari per calcolare indicatori chiave quali: la percentuale di confidenza media delle estrazioni, la frequenza dell'intervento umano per le correzioni, l'accuratezza strutturale del mapping dei dati e i tempi medi di elaborazione dei documenti.

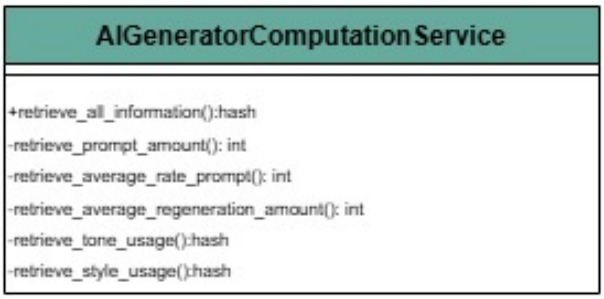
Attributi

- Nessuno

Metodi

- retrieve_all_information():hash
- retrieve_prompt_amount(): int
- retrieve_average_rate_prompt(): int
- retrieve_average_regeneration_amount(): int
- retrieve_tone_usage():hash
- retrieve_style_usage():hash

5.3.1.5 AiGeneratorComputationService



Classe concreta contenente la logica di business per la generazione delle statistiche relative all'AI-assistant. Si occupa di aggregare e calcolare le metriche di utilizzo dell'assistente, tra cui il volume totale delle generazioni, la valutazione media fornita dagli utenti, il tasso di rigenerazione dei prompt e la distribuzione percentuale dell'utilizzo dei vari toni e stili di comunicazione.

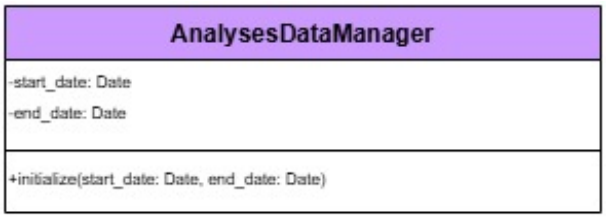
Attributi

- Nessuno

Metodi

- retrieve_all_information(): hash
- retrieve_average_confidence_value(): int
- retrieve_human_intervention_value(): int
- retrieve_mapping_accuracy(): int
- retrieve_average_time_analyses(): float

5.3.1.6 AnalysesDataManager



Interfaccia o classe base per il livello di persistenza (Data Access Object / Repository) dedicato alle analytics. Definisce le operazioni comuni per l'interrogazione del database al fine di estrarre lo storico delle interazioni, i log di sistema e i metadati senza esporre la complessità delle query SQL (o ORM) ai livelli superiori.

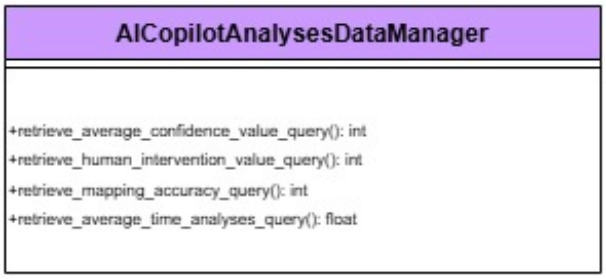
Attributi

- start_date: Date
- end_date: Date

Metodi

- initialize(start_date: Date, end_date: Date)

5.3.1.7 AiCopilotAnalysesDataManager



Implementazione specifica del Data Manager responsabile del recupero dei dati storici legati al modulo AI Co-Pilot. Interroga le tabelle del database pertinenti (es. storico dei documenti caricati, audit log delle modifiche manuali, timestamp di elaborazione) estraendo dataset filtrati che

verranno poi passati al *AiCopilotComputationService* per le elaborazioni statistiche.

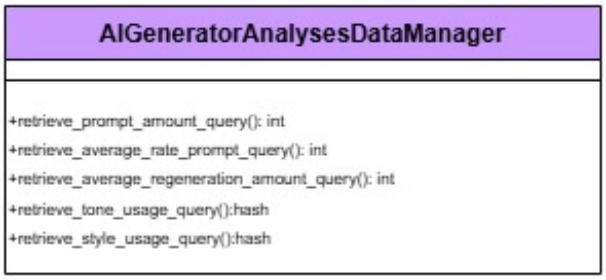
Attributi

- Nessuno

Metodi

- retrieve_prompt_amount_query(): int
- retrieve_average_rate_prompt_query(): int
- retrieve_average_regeneration_amount_query(): int
- retrieve_tone_usage_query():hash
- retrieve_style_usage_query():hash

5.3.1.8 AiGeneratorAnalysesDataManager



Implementazione specifica del Data Manager per il recupero dei dati relativi all'AI-assistant generativo. Si occupa di eseguire query sul database per estrarre lo storico dei prompt inviati, i parametri utilizzati (tono e stile), i feedback registrati (rating) e i conteggi delle richieste di rigenerazione, fornendo il set di dati grezzi al *AiGeneratorComputationService*.

Attributi

- Nessuno

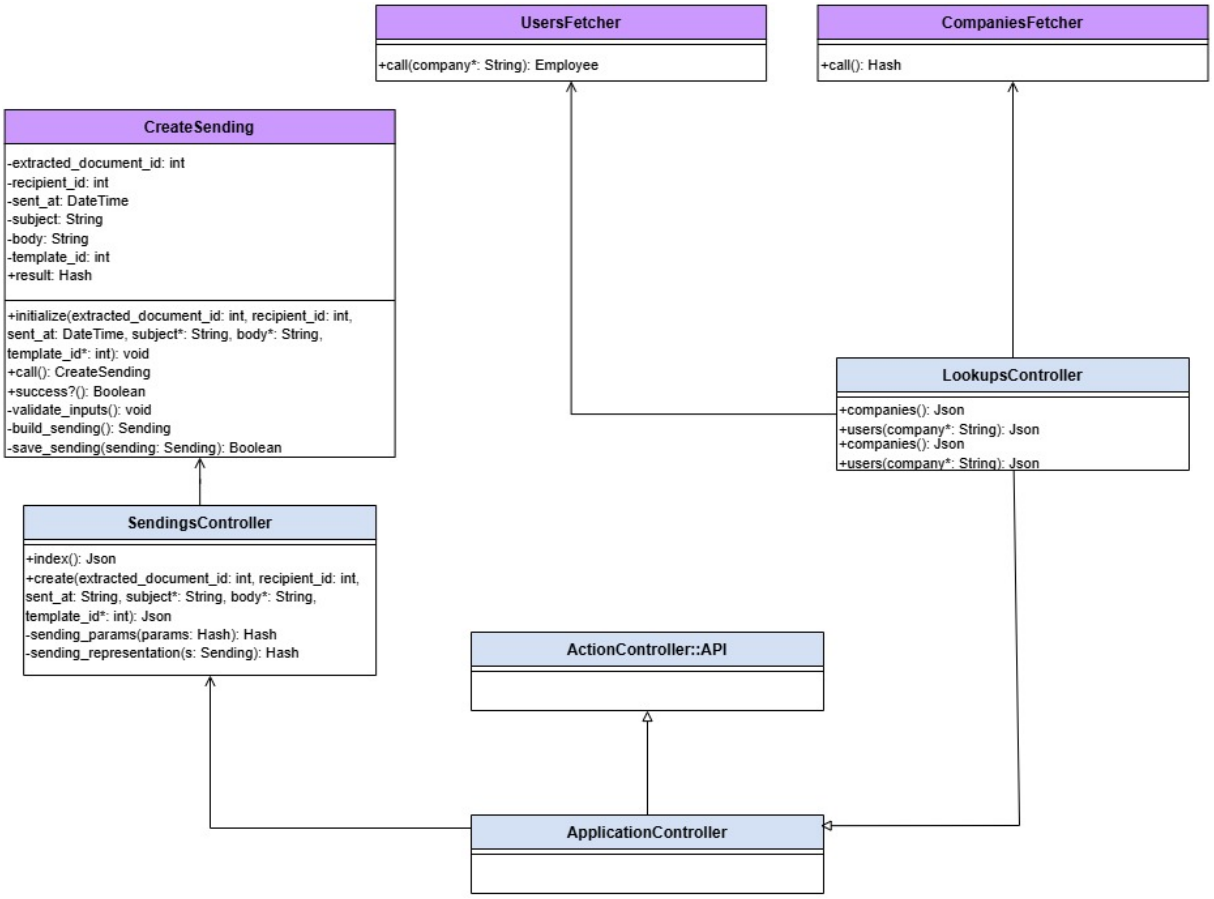
Metodi

- retrieve_average_confidence_value_query(): int
- retrieve_human_intervention_value_query(): int
- retrieve_mapping_accuracy_query(): int
- retrieve_average_time_analyses_query(): float

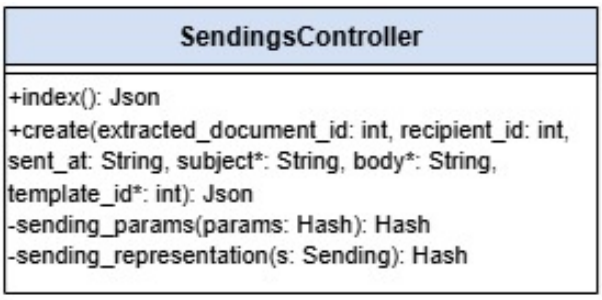
5.4 Classi condivise

Questa sezione include classi che, pur non appartenendo a un modulo specifico, forniscono funzionalità trasversali utilizzate sia dal modulo di AI Co-Pilot che da quello di AI-assistant generativo.

5.4.1 Diagramma delle classi blocco modulo Classi condivise



5.4.1.1 SendingsController



Questa classe di tipo Controller gestisce le richieste HTTP relative alla creazione e gestione delle "sendings", ovvero le comunicazioni inviate ai destinatari in seguito all'estrazione dei dati. Fornisce endpoint per visualizzare le sendings esistenti e per crearne di nuove, delegando la logica di business alla classe *CreateSending*. Inoltre, include metodi di utilità per formattare i parametri di invio e rappresentare le sendings in un formato adatto alla risposta JSON.

Attributi

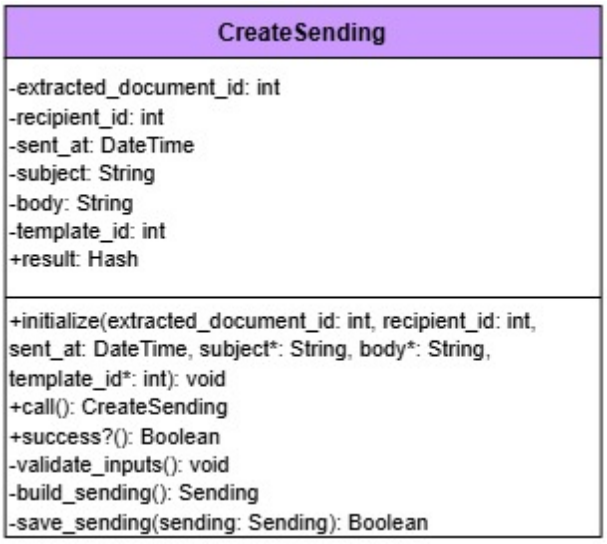
- Nessuno

Metodi

- index(): Json
- create(extracted_document_id: int, recipient_id: int, sent_at: String, subject*: String, body*: String, template_id*: int): Json
- sending_params(params: Hash): Hash
- sending_representation(s: Sending): Hash

5.4.1.2 CreateSending

Questa classe di tipo Command incapsula la logica necessaria per creare una nuova sending. Si occupa di validare i parametri di input, costruire l'oggetto Sending da salvare nel database e gestire eventuali errori durante il processo di creazione.



Attributi

- extracted_document_id: int
- recipient_id: int
- sent_at: DateTime
- subject: String
- body: String
- template_id: int
- result: Hash

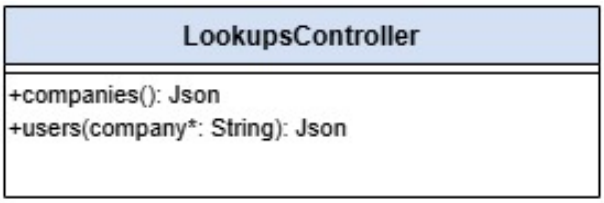
Metodi

- initialize(extracted_document_id: int, recipient_id: int, sent_at: DateTime, subject*: String, body*: String, template_id*: int): void
- call(): CreateSending
- success?(): Boolean
- validate_inputs(): void

- build_sending(): Sending
- save_sending(sending: Sending): Boolean

5.4.1.3 LookupsController

Questa classe di tipo Controller fornisce endpoint API per recuperare dati di riferimento condivisi, come la lista delle aziende e degli utenti associati a ciascuna azienda. Questi dati sono utilizzati sia dal modulo di AI Co-Pilot che da quello di AI-assistant generativo per popolare dropdown, assegnare responsabilità o filtrare le informazioni visualizzate. Il controller delega la logica di recupero dei dati a classi di servizio dedicate (es. *UsersFetcher* e *CompaniesFetcher*).



Attributi

- Nessuno

Metodi

- companies(): Json
- users(company*: String): Json

5.4.1.4 UsersFetcher

Questa classe di tipo Service Object è responsabile del recupero degli utenti associati a una specifica azienda. Fornisce un'interfaccia semplice per ottenere la lista degli employee da un database o da un servizio esterno, filtrando per azienda. Viene utilizzata principalmente dal *LookupsController* per popolare le dropdown di selezione degli utenti nel front-end.

Attributi

- Nessuno

Metodi

- call(company*: String): Employee

5.4.1.5 CompaniesFetcher

Questa classe di tipo Service Object è responsabile del recupero della lista delle aziende disponibili. Fornisce un'interfaccia semplice per ottenere l'elenco delle aziende da un database o da un servizio esterno, restituendo i dati in un formato adatto alla serializzazione JSON. Viene utilizzata principalmente dal *LookupsController* per popolare le dropdown di selezione delle aziende nel front-end.

Attributi

- Nessuno

Metodi

- call(): Hash

6 Descrizione architettura (Frontend)

7 Design Pattern

La progettazione e lo sviluppo del sistema si sono avvalsi di consolidati design pattern, sia implementati esplicitamente nella logica di dominio, sia ereditati intrinsecamente dal framework Ruby on Rails. L'adozione di tali paradigmi ha garantito modularità, manutenibilità ed elevata testabilità dell'intero applicativo.

Il backend è strutturato attorno a un'architettura a servizi stratificata: i *Controller* gestiscono il routing HTTP; i *Command* incapsulano le operazioni di business; i *Service Object* orchestrano pipeline complesse; i *Repository* astraggono la persistenza; il *Container* governa l'iniezione delle dipendenze. I componenti *Job* delegano l'elaborazione pesante a code asincrone notificando il frontend in tempo reale tramite ActionCable.

La tabella 16 riassume tutti i pattern documentati in questa sezione.

Pattern	Categoria	GoF / POEAA	Dove applicato
Service Object	Comportamentale	—	`ProcessDataItem`, `ProcessSplitRun`, `ProcessGenericFile`, `AIGeneratorService`
Command	Comportamentale	GoF	`InitializeProcessing`, `InitializeFileProcessing`, `ReassignExtractedRange`
Repository	Architetturale	POEAA	`DataItemRepository`, `SplitRunRepository`
Container / DI	Creazionale	—	`DocumentProcessing::Container`, `AiGeneratorContainer`
Strategy	Comportamentale	GoF	`file_processor` (CSV / Image)
Factory Method	Creazionale	GoF	Metodi factory nel Container
Presenter	Strutturale	—	`ExtractedDocumentPresenter`
Serializer	Strutturale	—	`PostSerializer`, `ToneSerializer`, `StyleSerializer`
Observer	Comportamentale	GoF	`ActionCableNotifier`, `AiJobOrchestrator`
Active Record	Architetturale	POEAA	Tutti i modelli Rails
MVC	Architetturale	—	Intera applicazione Rails

Tabella 16: Riepilogo dei design pattern adottati

7.1 Pattern implementati esplicitamente

7.1.1 Service Object

Intent: Incapsulare un'operazione di business complessa in una classe dedicata con un'unica responsabilità, esponendo un'interfaccia uniforme `call(...)`, o, in alternativa, un set di metodi pubblici che rappresentano le fasi dell'operazione.

Il pattern *Service Object* costituisce l'ossatura della logica applicativa. Ogni servizio ha una responsabilità precisa e non porta stato persistente tra invocazioni distinte.

Esempi:

7.1.1.1 ProcessDataItem

Orchestratore centrale della pipeline OCR: coordina riconoscimento ottico dei caratteri, estrazione dati tramite LLM, calcolo della confidenza, risoluzione del destinatario e persistenza del risultato.

Listing 1: ProcessDataItem – struttura del metodo `call`

```
# app/services/document_processing/process_data_item.rb
module DocumentProcessing
  class ProcessDataItem
    def initialize(
      ocr_service:, data_extractor:, recipient_resolver:,
      confidence_calculator_factory:, data_item_repository:,
      notifier:, metadata_builder:, file_storage:
```

```
    )
      @ocr_service              = ocr_service
      @data_extractor           = data_extractor
      @recipient_resolver       = recipient_resolver
      @confidence_calculator_factory = confidence_calculator_factory
      @data_item_repository     = data_item_repository
      @notifier                 = notifier
      @metadata_builder         = metadata_builder
      @file_storage             = file_storage
    end

    def call(file_path:, job_id:, processing_item_id: nil,
            extracted_document_id: nil)
      item    = @data_item_repository.find_processing_item(processing_item_id)
      document = @data_item_repository.find_extracted_document(extracted_document_id)

      return if @data_item_repository.terminal_item?(item)

      @data_item_repository.mark_item_in_progress!(item)

      # 1. OCR
      ocr_result = @ocr_service.extract_text(file_path)
      # 2. Estrazione strutturata via LLM
      extracted = @data_extractor.extract(ocr_result.text)
      # 3. Confidenza
      confidence = @confidence_calculator_factory.call(
        ocr_lines: ocr_result.lines, llm_result: extracted
      ).global_confidence
      # 4. Risoluzione destinatario
      resolution = @recipient_resolver.resolve(extracted.recipient)
      # 5. Persistenza e notifica
      @data_item_repository.mark_item_done!(item: item, resolution: resolution)
      @notifier.broadcast(job_id, { event: "document_processed", ... })
    rescue StandardError => e
      @data_item_repository.mark_item_failed(item: item, error_message: e.message)
      @notifier.broadcast(job_id, { event: "error", message: e.message })
    ensure
      @file_storage.delete(file_path) if @file_storage.exist?(file_path)
    end
  end
end
```

7.1.1.2 ProcessSplitRun

Gestisce la fase di splitting di un PDF: divide il documento in sotto-PDF, crea i record `ProcessingItem` / `ExtractedDocument` associati e accoda un job `DataExtractionJob` per ciascuna porzione.

7.1.1.3 ProcessGenericFile

Gestisce file non-PDF (CSV, immagini) tramite il pattern Strategy (si veda la sezione 7.1.5): seleziona il processore appropriato in base al `file_kind` e applica gli override utente con confidenza 1.0.

7.1.2 Command

Intent (GoF): Incapsulare una richiesta come oggetto, parametrizzare i client con operazioni diverse e supportare operazioni reversibili o accodabili.

I Command centralizzano le operazioni di business esposte dai Controller, restituendo sempre un hash di risultato strutturato e isolando la gestione degli errori dalla presentazione HTTP.

Listing 2: InitializeProcessing – validazione e avvio pipeline

```
# app/services/document_processing/commands/initialize_processing.rb
module DocumentProcessing
  module Commands
    class InitializeProcessing
      def initialize(upload_manager:, file_storage:,
                     data_item_repository:, job_class: PdfSplitJob)
        @upload_manager    = upload_manager
        @file_storage      = file_storage
        @data_item_repository = data_item_repository
        @job_class         = job_class
      end

      def call(file:, category: nil, company: nil,
               department: nil, competence_period: nil, employee: nil)
        checksum    = @upload_manager.compute_checksum(file)
        existing    = UploadedDocument.find_by(checksum: checksum)
        return already_exists_result(existing) if existing

        storage_path = @upload_manager.persist_source_pdf(file)
        uploaded_doc = UploadedDocument.create!(
          original_filename: file.original_filename,
          storage_path:      storage_path,
          checksum:          checksum,
          file_kind:         "pdf",
          override_company:  company,
          override_department: department
        )
        run = ProcessingRun.create!(
          job_id: SecureRandom.uuid, status: "queued",
          uploaded_document: uploaded_doc
        )
        @job_class.perform_later(storage_path, run.job_id)

        { status: "queued", job_id: run.job_id,
          uploaded_document_id: uploaded_doc.id, message: "Avviato" }
      rescue UploadManager::ValidationError => e
        { ok: false, error: :validation, message: e.message }
      rescue UploadManager::PersistenceError
        { ok: false, error: :persistence, message: "Errore nel salvataggio" }
      end
    end
  end
end
```

Listing 3: ReassignExtractedRange – riprocessamento con nuovo range

```
# app/services/document_processing/commands/reassign_extracted_range.rb
def call(extracted_document_id:, page_start:, page_end:)
  doc = ExtractedDocument.find(extracted_document_id)
```

```
  unless page_start.is_a?(Integer) && page_end.is_a?(Integer) &&
        page_start > 0 && page_end >= page_start &&
        page_end <= doc.uploaded_document.page_count
    return { ok: false, error: :validation,
             message: "Range pagine non valido" }
  end

  temp_path = @page_range_pdf_service.build_temp_pdf(
    page_start: page_start, page_end: page_end
  )
  doc.update!(status: "queued", page_start: page_start, page_end: page_end)
  DataExtractionJob.perform_later(temp_path, {
    job_id: doc.uploaded_document.processing_runs.last.job_id,
    extracted_document_id: doc.id
  })
  { ok: true, extracted_document_id: doc.id,
    page_start: page_start, page_end: page_end,
    message: "Riassegnazione completata" }
end
```

I tre Command del sistema sono riepilogati in tabella 18.

Command	Responsabilità	Job accodato
`InitializeProcessing`	Valida PDF, crea `UploadedDocument` e `ProcessingRun`	`PdfSplitJob`
`InitializeFileProcessing`	Valida CSV/immagine, crea record	`GenericFileProcessingJob`
`ReassignExtractedRange`	Aggiorna range pagine, riaccoda estrazione	`DataExtractionJob`

Tabella 18: Command e rispettivi job accodati

7.1.3 Repository

Intent (POEAA – Fowler): Mediare tra il dominio e il livello di mapping dati usando un'interfaccia simile a una collezione per accedere agli oggetti del dominio.

I Repository centralizzano tutte le transizioni di stato e le query di persistenza, isolando il resto dell'applicazione da ActiveRecord. Rails ha uno degli ORM piú completi sul mercato e per query semplici è possibile utilizzare direttamente i modelli, in quanto le operazioni sono native e non ha senso introdurre un ulteriore livello di astrazione che non aggiunge valore. Tuttavia, per operazioni più complesse (es. transizioni di stato con lock, query aggregate) è consigliabile incapsulare la logica in un Repository dedicato, evitando duplicazioni e garantendo la correttagestione della concorrenza, permettendo di separare nettamente la logica di dominio dalla persistenza.

Listing 4: DataItemRepository – transizioni di stato con pessimistic locking

```
# app/services/document_processing/persistence/data_item_repository.rb
module DocumentProcessing
  module Persistence
    class DataItemRepository

      # Trova un ProcessingItem per id
      def find_processing_item(id)
        ProcessingItem.find(id)
```

```
      end

      # Transizione atomica in_progress (lock pessimistico)
      def mark_item_in_progress!(item)
        item.with_lock do
          item.reload
          return if item.done? || item.failed?
          item.update!(status: "in_progress")
        end
      end

      # Transizione done con aggiornamento employee
      def mark_item_done!(item:, resolution:)
        item.with_lock do
          item.update!(status: "done")
          if resolution.matched?
            item.extracted_document
                .update!(matched_employee: resolution.employee)
          end
        end
      end

      # Aggiorna il progresso del ProcessingRun
      def update_progress!(run)
        run.with_lock do
          total = run.processing_items.count
          done  = run.processing_items.where(status: %w[done failed]).count
          pct   = total.positive? ? (done.to_f / total * 100).round : 0
          run.update!(progress_percentage: pct)
          run.update!(status: "completed") if done == total
        end
      end

      def transaction(&block)
        ApplicationRecord.transaction(&block)
      end
    end
  end
end
```

Il *Pessimistic Locking* (`with_lock`) è fondamentale per garantire la correttezza delle transizioni di stato quando più job elaborano documenti in parallelo: senza lock, due worker potrebbero leggere lo stesso stato `queued` e procedere simultaneamente.

Listing 5: SplitRunRepository – creazione artefatti di splitting

```
# app/services/document_processing/persistence/split_run_repository.rb
def create_split_artifacts!(run:, split_results:)
  split_results.each_with_index do |result, idx|
    extracted = ExtractedDocument.create!(
      uploaded_document: run.uploaded_document,
      sequence:  idx + 1,
      page_start: result[:page_start],
      page_end:  result[:page_end],
      status:    "queued"
    )
    ProcessingItem.create!(
      processing_run:  run,
      extracted_document: extracted,
```

```
      sequence:        idx + 1,
      status:          "queued"
    )
  end
end
```

7.1.4 Container e Dependency Injection

Intent: Centralizzare la creazione e la composizione delle dipendenze in un oggetto dedicato (IoC Container), permettendo ai client di ricevere le dipendenze senza istanziarle direttamente.

7.1.4.1 DocumentProcessing::Container

Il Container è il cuore dell'architettura: gestisce l'intero grafo di dipendenze del modulo OCR, inizializzando i servizi con lazy memoization e supportando l'override di qualsiasi componente per i test.

Listing 6: DocumentProcessing::Container – composizione delle dipendenze

```
# app/services/document_processing/container.rb
module DocumentProcessing
  class Container
    def initialize(
      ocr_service_class:     DocumentProcessing::OcrService,
      data_extractor_class:  DocumentProcessing::DataExtractor,
      pdf_splitter_class:    DocumentProcessing::PdfSplitter,
      job_class:             PdfSplitJob,
      notifier_class:        DocumentProcessing::ActionCableNotifier,
      **overrides
    )
      @ocr_service_class  = ocr_service_class
      @data_extractor_class = data_extractor_class
      @pdf_splitter_class = pdf_splitter_class
      @job_class          = job_class
      @notifier_class     = notifier_class
      @overrides          = overrides
    end

    # Memoizzazione lazy: il client Textract viene creato
    # solo al primo accesso
    def ocr_service
      @ocr_service ||= @ocr_service_class.new(
        client: Aws::Textract::Client.new(region: aws_region)
      )
    end

    # Factory Method: seleziona il processore corretto per tipo file
    def file_processor(file_kind)
      case file_kind.to_s
      when "csv"  then csv_processor
      when "image" then image_processor
      else raise ArgumentError, "file_kind non supportato: #{file_kind}"
      end
    end

    # Composizione complessa: process_data_item_service aggrega
```

```
    # 8 dipendenze diverse
    def process_data_item_service
      @process_data_item_service ||= ProcessDataItem.new(
        ocr_service:              ocr_service,
        data_extractor:           data_extractor,
        recipient_resolver:       recipient_resolver,
        confidence_calculator_factory: method(:confidence_calculator),
        data_item_repository:     data_item_repository,
        notifier:                 notifier,
        metadata_builder:         metadata_builder,
        file_storage:             file_storage
      )
    end

    def initialize_processing_command
      Commands::InitializeProcessing.new(
        upload_manager:     upload_manager,
        file_storage:       file_storage,
        data_item_repository: data_item_repository,
        job_class:          @job_class
      )
    end

    private

    def aws_region
      ENV.fetch("AWS_REGION", "eu-west-1")
    end
  end
end
```

Questo approccio consente di testare qualsiasi servizio sostituendo una singola dipendenza tramite override del costruttore, senza modificare il codice di produzione:

Listing 7: Utilizzo del Container nei test – override delle dipendenze

```
# test/controllers/documents_controller_test.rb
def fake_container(overrides = {})
  init_cmd = fake_command(
    overrides.fetch(:init_result,
      { status: "queued", job_id: "j1", uploaded_document_id: 1 })
  )
  Object.new.tap do |c|
    c.define_singleton_method(:initialize_processing_command) { init_cmd }
    c.define_singleton_method(:file_storage) {
      overrides.fetch(:file_storage, DocumentProcessing::Persistence::FileStorage.new)
    }
    c.define_singleton_method(:db_manager) {
      overrides.fetch(:db_manager, real_db_manager)
    }
  end
end

# Nel test: il Container reale viene sostituito con il fake
stub_new(DocumentProcessing::Container, fake_container) do
  post split_documents_path, params: { pdf: "dummy" }
end
```

7.1.4.2 AiGenerator::AiGeneratorContainer

Container analogo per il modulo di generazione AI: gestisce i servizi Bedrock per testo e immagini, il data manager e la factory dei parametri.

Listing 8: AiGeneratorContainer – servizi Bedrock con configurazione da ENV

```
# app/services/ai_generator/ai_generator_container.rb
module AiGenerator
  class AiGeneratorContainer
    def aiGeneratorService
      @aiGeneratorService ||= AIGeneratorService.new(
        textGenerator:    textGenerator,
        imgGenerator:     imgGenerator,
        setterFactory:    setterFactory,
        aiGeneratorDataManager: aiGeneratorDataManager,
        validationService: AiGeneratorValidationService.new
      )
    end

    private

    def textGenerator
      @textGenerator ||= TextGeneratorService.new(
        client: Aws::BedrockRuntime::Client.new(region: bedrock_region)
      )
    end

    def imgGenerator
      @imgGenerator ||= ImageGeneratorService.new(
        client: Aws::BedrockRuntime::Client.new(region: bedrock_region)
      )
    end

    def bedrock_region
      ENV.fetch("BEDROCK_REGION", "us-east-1")
    end
  end
end
```

7.1.5 Strategy

Intent (GoF): Definire una famiglia di algoritmi, incapsulare ciascuno e renderli intercambiabili. Lo Strategy permette di variare l'algoritmo indipendentemente dai client che lo utilizzano.

Il metodo `file_processor` del Container implementa la selezione dinamica della strategia di elaborazione in base al tipo di file:

Listing 9: Strategy – selezione del processore per tipo file

```
# In DocumentProcessing::Container
def file_processor(file_kind)
  case file_kind.to_s
  when "csv"  then csv_processor # Strategia CSV
  when "image" then image_processor # Strategia immagine
  else raise ArgumentError, "file_kind non supportato: #{file_kind}"
  end
end
```

```
def csv_processor
  @csv_processor ||= CsvProcessor.new(
    recipient_resolver: recipient_resolver,
    metadata_builder: metadata_builder
  )
end

def image_processor
  @image_processor ||= ImageProcessor.new(
    ocr_service:      ocr_service,
    data_extractor:   data_extractor,
    recipient_resolver: recipient_resolver
  )
end
```

Entrambi i processori condividono la stessa interfaccia `call(file_path)` e restituiscono un hash uniforme con `metadata`, `confidence`, `recipient` e `ocr_text`. Il chiamante (`ProcessGenericFile`) non conosce quale strategia stia usando:

Listing 10: ProcessGenericFile – polimorfismo tramite Strategy

```
# Il servizio riceve il processore dal Container: non conosce
# se e' CSV o immagine
def call(file_path:, job_id:, uploaded_document_id:, ...)
  uploaded = UploadedDocument.find(uploaded_document_id)
  # file_processor e' gia' la strategia corretta
  result = @file_processor.call(file_path)

  # Override utente: confidenza 1.0 (certezza massima)
  result[:metadata].merge!(override_company: override_company) if override_company
  create_extracted_document(uploaded, result, ...)
end
```

7.1.6 Factory Method

Intent (GoF): Definire un'interfaccia per la creazione di un oggetto, lasciando alle sottoclassi (o ai metodi specializzati) la decisione su quale classe istanziare.

I metodi del Container agiscono come Factory Method: centralizzano la logica di costruzione di oggetti complessi e le loro dipendenze. Il chiamante invoca il metodo senza conoscere la classe concreta istanziata:

Listing 11: Factory Method nel Container – costruzione lazy con memoization

```
# Il Controller ottiene il Command senza conoscere come e' costruito
def initialize_processing_command # <- Factory Method
  Commands::InitializeProcessing.new(
    upload_manager:     upload_manager, # altra factory
    file_storage:       file_storage,
    data_item_repository: data_item_repository,
    job_class:          @job_class
  )
end

# Chiamata nel Controller: nessuna dipendenza diretta
def split
  result = initialize_processing_command.call(
```

```
    file:    params[:pdf],
    company: params[:company],
    category: params[:category]
  )
  render json: result
end
```

7.1.7 Presenter e Serializer

Intent: Separare la logica di presentazione/formattazione dal modello dati, producendo rappresentazioni JSON coerenti e disaccoppiate dai dettagli di persistenza.

7.1.7.1 ExtractedDocumentPresenter

Trasforma un record `ExtractedDocument` (con le sue relazioni) nella struttura JSON restituita dall'API, generando anche l'URL di download del PDF:

Listing 12: ExtractedDocumentPresenter – trasformazione modello in JSON

```
# app/services/document_processing/presenters/extracted_document_presenter.rb
module DocumentProcessing
  module Presenters
    class ExtractedDocumentPresenter
      include Rails.application.routes.url_helpers

      def initialize(extracted_document)
        @doc = extracted_document
      end

      def as_json(*)
        {
          id:                  @doc.id,
          uploaded_document_id: @doc.uploaded_document_id,
          sequence:            @doc.sequence,
          status:              @doc.status,
          page_start:          @doc.page_start,
          page_end:            @doc.page_end,
          metadata:            @doc.metadata,
          recipient:           @doc.recipient,
          confidence:          @doc.confidence,
          matched_employee:    employee_json,
          error_message:       @doc.error_message,
          processed_at:        @doc.processed_at,
          process_time_seconds: @doc.process_time_seconds,
          pdf_download_url:    pdf_url
        }
      end

      private

      def employee_json
        return nil unless @doc.matched_employee
        e = @doc.matched_employee
        {
          id:           e.id,
          name:         e.name,
```

```
        email:        e.email,
        employee_code: e.username
      }
    end

    def pdf_url
      extracted_pdf_document_path(id: @doc.id)
    end
  end
 end
end
```

7.1.7.2 Serializer per collezioni

I Serializer producono la struttura JSON delle collezioni REST con chiavi camelCase coerenti con il frontend:

Listing 13: PostSerializer – serializzazione con join eagerly loaded

```
# app/serializers/post_serializer.rb
module PostSerializer
  def self.serialize_collection(posts)
    {
      posts: posts.map do |post|
        gd = post.generated_datum
        {
          id:            post.id,
          title:         post.title,
          PostText:      post.body_text,
          imgPath:       post.img_path,
          dateTime:      post.date_time,
          generatedDatumId: post.generated_datum_id,
          toneId:        gd&.tone_id,
          toneName:      gd&.tone&.name,
          styleId:       gd&.style_id,
          styleName:     gd&.style&.name,
          companyId:     gd&.company_id,
          companyName:   gd&.company&.name,
          rating:        gd&.rating
        }
      end
    }
  end
end
```

7.1.8 Observer (ActionCable Notifier)

Intent (GoF): Definire una dipendenza uno-a-molti tra oggetti in modo che, quando un oggetto cambia stato, tutti i dipendenti vengano notificati e aggiornati automaticamente.

7.1.8.1 ActionCableNotifier

Broadcaster WebSocket che funge da Subject: notifica il frontend in tempo reale a ogni cambiamento di stato del pipeline di elaborazione:

Listing 14: ActionCableNotifier – broadcast di eventi sul canale dedicato

```
# app/services/document_processing/action_cable_notifier.rb
module DocumentProcessing
  class ActionCableNotifier
    # Ogni job di elaborazione ha il proprio canale isolato
    def broadcast(job_id, data)
      ActionCable.server.broadcast(
        "document_processing:#{job_id}",
        data
      )
    end
  end
end

# Esempio di eventi trasmessi:
# { event: "split_completed",    total_documents: 5 }
# { event: "document_processed", sequence: 2, status: "done" }
# { event: "processing_completed", processed: 5 }
# { event: "error",              message: "OCR fallita" }
```

7.1.8.2 AiJobOrchestrator

Orchestratore degli stati del job AI: coordina creazione, avvio, completamento e fallimento della generazione, notificando il frontend sul canale `generation_channel`:

Listing 15: AiJobOrchestrator – ciclo di vita del job di generazione

```
# app/services/ai_generator/ai_job_orchestrator.rb
module AiGenerator
  class AiJobOrchestrator
    def orchestrate(params)
      datum = GeneratedDatum.create!(params.merge(status: "queued"))
      AiGeneratorJob.perform_later(datum.id)
      datum
    end

    def signal_process_start(generation_id)
      ActionCable.server.broadcast("generation_channel",
        { status: "processing", generationId: generation_id })
    end

    def complete(generation_id)
      datum = GeneratedDatum.find(generation_id)
      ActionCable.server.broadcast("generation_channel", {
        status:      "completed",
        generationId: generation_id,
        imageUrl:    rails_blob_url(datum.generated_image),
        title:       datum.title,
        bodyText:    datum.body_text
      })
    end

    def signal_failure(generation_id, error_message)
      ActionCable.server.broadcast("generation_channel",
        { status: "failed", generationId: generation_id,
          error: error_message })
    end
```

```
  end
end
```

7.2 Pattern ereditati dal framework

7.2.1 Model-View-Controller (MVC)

L'architettura dell'intera applicazione è fondata sul pattern MVC imposto da Ruby on Rails:

- **Model**: gestisce la logica dei dati, le validazioni e le associazioni tra entità (es. `ExtractedDocument`, `UploadedDocument`, `Company`).
- **View**: in questo ecosistema API-only, la view è rappresentata da un risultato JSON serializzato.
- **Controller**: orchestra il flusso, validando i parametri HTTP e delegando ai Command o ai Service Object la logica di business.

Non si tratta del tradizionale approccio MVC basato su template HTML generati lato server — in questo caso, infatti, la "view" è rappresentata da dati in formato JSON — ma la struttura di base rimane comunque riconoscibile. I controller continuano a svolgere il ruolo di intermediari tra le richieste HTTP e la logica applicativa, che è incapsulata nei servizi e nei modelli.
Ci troviamo però di fronte a un approccio più moderno, caratterizzato da una netta separazione tra backend e frontend. Il backend si limita a esporre API RESTful, senza occuparsi del rendering o della presentazione dei dati. Questo consente di ottenere un'architettura più modulare, scalabile e manutenibile.
Il rendering viene invece delegato ad Angular, che gestisce la visualizzazione lato client (client-side rendering), evitando di sovraccaricare il server. Questo si differenzia dal classico MVC, in cui il rendering avviene lato server (server-side rendering).

Questo modello dunque rappresenta oggi uno standard per applicazioni sviluppate con Ruby on Rails e offre diversi vantaggi: maggiore scalabilità, flessibilità nella scelta del frontend, interfacce più reattive grazie al paradigma delle Single Page Application (SPA), distribuzione del carico sul client e possibilità di riutilizzare il backend per applicazioni mobile o integrazioni con servizi di terze parti.

Listing 16: DocumentsController – Controller MVC che delega ai Command

```
# app/controllers/documents_controller.rb
class DocumentsController < ActionController::Base
  skip_before_action :verify_authenticity_token

  # POST /documents/split
  def split
    result = initialize_processing_command.call(
      file:           params[:pdf],
      company:        params[:company],
      category:       params[:category],
      competence_period: params[:competence_period]
    )
    if result.is_a?(Hash) && result[:ok] == false
      return render_error(result[:message])
    end
    render json: {
      status:            result[:status],
      job_id:            result[:job_id],
```

```
      uploaded_document_id: result[:uploaded_document_id]
    }
  rescue StandardError => e
    render_error("Errore: #{e.message}")
  end

  # DELETE /documents/uploads/:id
  def destroy_upload
    uploaded = UploadedDocument.find(params[:id])
    file_storage.delete(uploaded.storage_path) if
      file_storage.exist?(uploaded.storage_path)
    uploaded.destroy! # cascade -> ExtractedDocument rimossi
    render json: { status: "ok", message: "Documento eliminato" }
  rescue ActiveRecord::RecordNotFound
    render json: { status: "error",
                   message: "Documento sorgente non trovato" },
           status: :not_found
  end

  private

  # Factory Method: il Container costruisce il Command
  def initialize_processing_command
    document_processing_container.initialize_processing_command
  end

  def document_processing_container
    @document_processing_container ||= DocumentProcessing::Container.new
  end
end
```

7.2.2 Active Record

Ogni classe Modello mappa una tabella del database (pattern *Active Record* di Martin Fowler), nascondendo la complessità SQL e fornendo un'interfaccia orientata agli oggetti per operazioni CRUD, relazioni e validazioni:

Listing 17: Modelli principali – associazioni, validazioni e predicate methods

```
# app/models/uploaded_document.rb
class UploadedDocument < ApplicationRecord
  has_many :extracted_documents, dependent: :destroy # cascade delete
  belongs_to :employee, optional: true

  validates :original_filename, :storage_path, :checksum, presence: true
  validates :checksum, uniqueness: true
  validates :file_kind, inclusion: { in: %w[pdf csv image] }, allow_nil: true
end

# app/models/extracted_document.rb
class ExtractedDocument < ApplicationRecord
  belongs_to :uploaded_document
  belongs_to :matched_employee, class_name: "User", optional: true

  STATUSES = %w[queued in_progress done failed sent validated].freeze

  validates :sequence, :page_start, :page_end, presence: true
```

```
  validates :status, inclusion: { in: STATUSES }, allow_nil: true
  validate :end_page_must_be_after_start

  # Predicate methods per leggibilita'
  def done?     = status == "done"
  def failed?   = status == "failed"
  def validated? = status == "validated"

  private
  def end_page_must_be_after_start
    return if page_start.blank? || page_end.blank?
    errors.add(:page_end, "deve essere >= page_start") if page_end < page_start
  end
end

# app/models/employee.rb
class Employee < ApplicationRecord
  belongs_to :user
  belongs_to :company
end

# app/models/company.rb
class Company < ApplicationRecord
  has_many :tones,        dependent: :destroy
  has_many :styles,       dependent: :destroy
  has_many :generated_data, dependent: :destroy
  validates :description, length: { maximum: 500 }
end
```

Le associazioni `dependent: :destroy` implementano implicitamente la gestione delle chiavi esterne a livello applicativo: la cancellazione di un `UploadedDocument` propaga automaticamente la rimozione di tutti gli `ExtractedDocument` collegati.

7.2.3 Observer tramite Callbacks

Rails implementa il pattern Observer attraverso il meccanismo dei *callbacks* legati al ciclo di vita dei modelli. Nel progetto, il framework gestisce automaticamente i timestamp `created_at` e `updated_at` ad ogni operazione di persistenza. Il pattern è esteso esplicitamente per il modello `ProcessingRun`:

Listing 18: Callback Rails – aggiornamento automatico dello stato

```
# app/models/processing_run.rb
class ProcessingRun < ApplicationRecord
  belongs_to :uploaded_document, optional: true
  has_many :processing_items, dependent: :destroy

  STATUSES = %w[queued splitting processing completed failed].freeze

  validates :job_id, presence: true, uniqueness: true
  validates :status, inclusion: { in: STATUSES }

  # Callback: aggiorna started_at al primo avvio
  before_save :set_started_at, if: -> { status_changed? && status == "processing" }

  private
  def set_started_at
    self.started_at ||= Time.current
```

```
  end
end
```

7.3 Database

7.3.1 Schema ER

7.3.2 Schema ERD (Entity-Relationship Diagram)

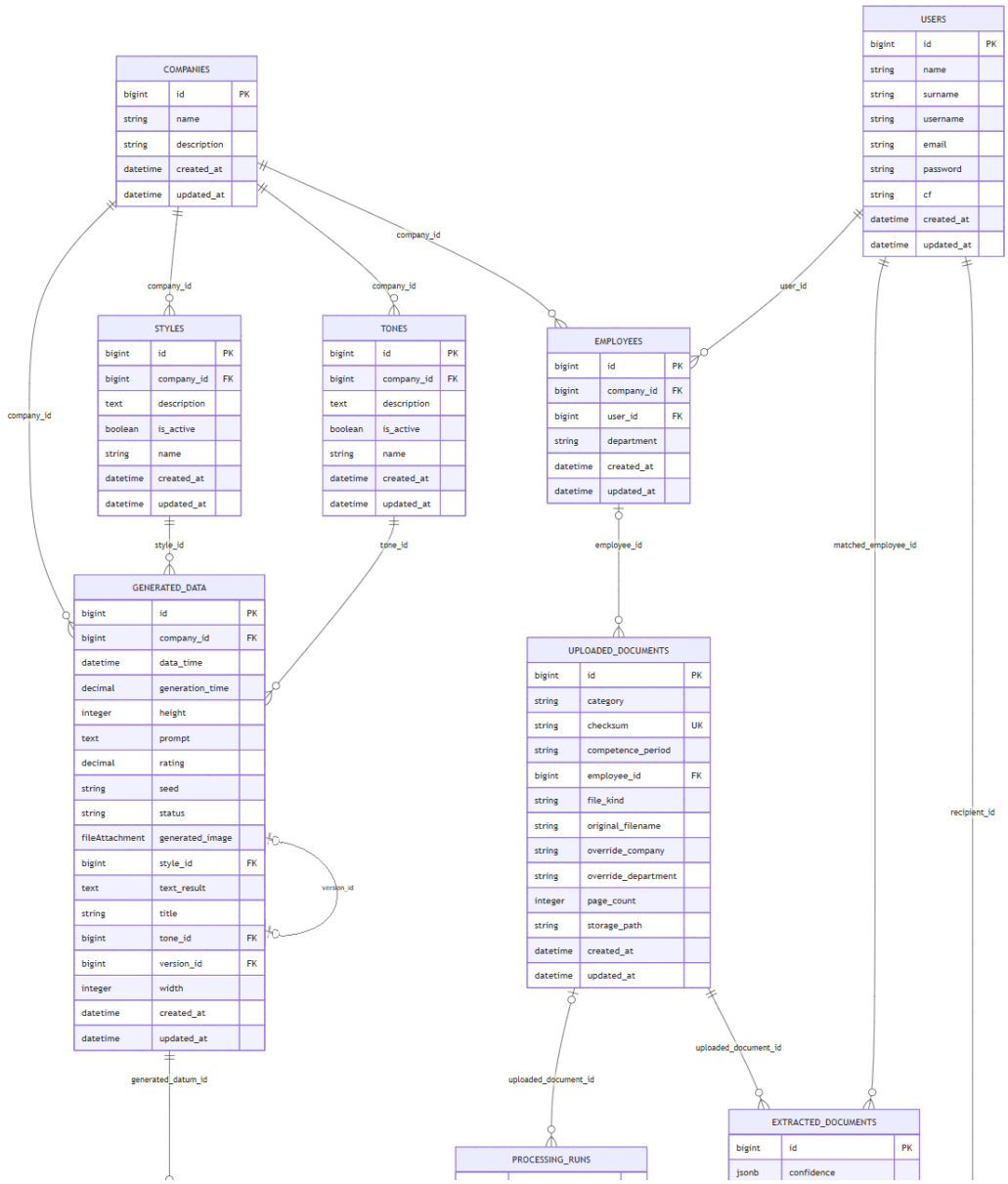


Figura 17: Schema ERD1

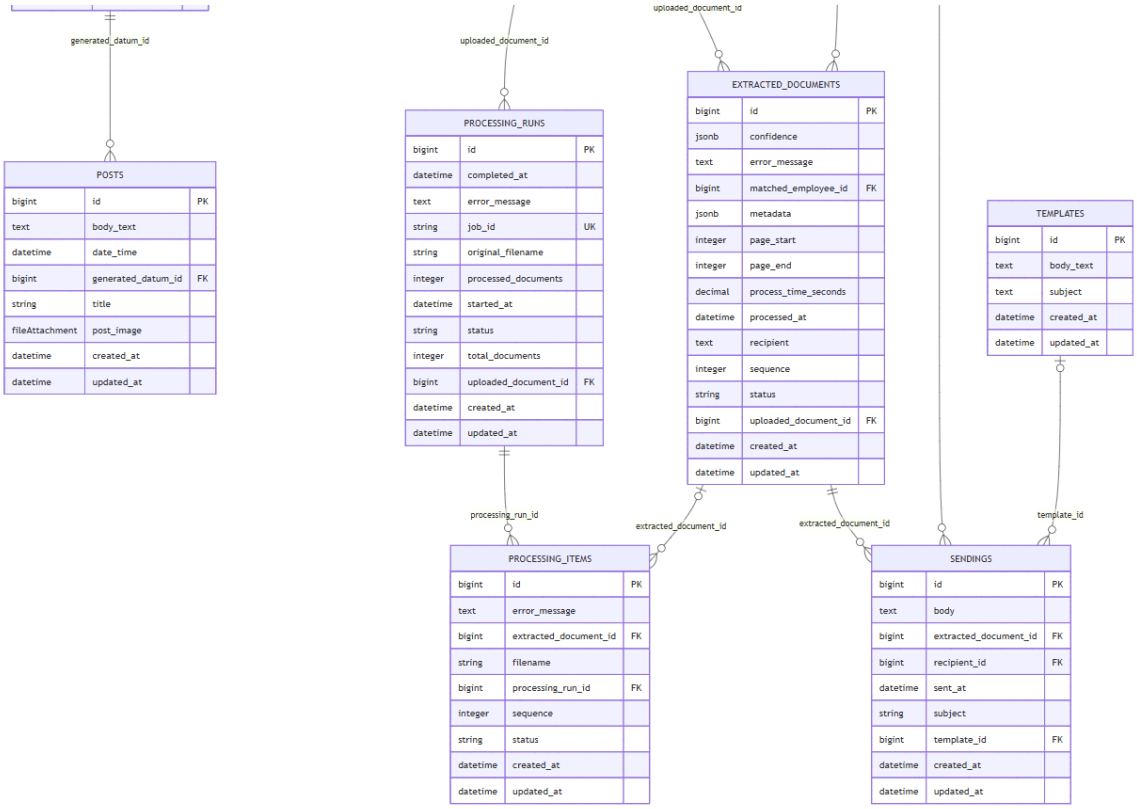


Figura 18: Schema ERD2

7.3.3 Descrizione tabelle

7.3.3.1 User

Costituisce l'entità anagrafica principale del sistema e gestisce l'accesso all'applicativo. Memorizza i dati personali dell'utente (codice fiscale, nome, cognome) e le credenziali di autenticazione (username, email, password). In un'architettura API-only, questa tabella è il punto di riferimento per l'emissione e la convalida dei token di sicurezza.

7.3.3.2 Company

Rappresenta l'organizzazione, l'azienda cliente o la divisione aziendale all'interno del sistema. Funge da aggregatore principale a cui vengono associati sia i dipendenti sia i parametri di personalizzazione per la generazione dei contenuti (toni e stili).

7.3.3.3 Employee

Funge da tabella di raccordo (join table con attributi) per mappare la relazione molti-a-molti tra *User* e *Company*. Permette di contestualizzare un utente all'interno di una specifica azienda, assegnandogli un dipartimento di competenza (*Department*).

7.3.3.4 Tones

Tabella di dizionario dedicata al modulo AI Generator. Definisce le direttive sul "tono di voce" (es. Professionale, Amichevole) da fornire in input ai modelli generativi. Ogni tono è vincolato a

una specifica azienda (*CompanyId*), garantendo una personalizzazione della generazione isolata per ogni tenant.

7.3.3.5 Style

Simile all'entità *Tones*, questa tabella definisce gli stili di scrittura o formattazione (es. Conciso, Dettagliato). Anche in questo caso, la relazione con *Company* permette di compartimentare i preset stilistici a livello aziendale.

7.3.3.6 GeneratedDatum

È l'entità centrale del modulo AI Generator. Traccia in modo esaustivo ogni interazione con il modello di intelligenza artificiale, persistendo l'input (*Prompt*), l'output testuale (*TextResult*) o visivo (*ImgPath*, *Seed*, *Height*, *Width*), le performance (*GenerationTime*) e il feedback utente (*Rating*). Presenta una chiave esterna autoreferenziale (*Versione*) cruciale per il tracciamento delle rigenerazioni successive di uno stesso contenuto base.

7.3.3.7 Post

Entità destinata a rappresentare l'utilizzo finale di un contenuto generato dal sistema. Collega direttamente un output testuale e/o grafico approvato proveniente da *GeneratedDatum* alle logiche di pubblicazione o schedulazione esterne, memorizzandone titolo, corpo del testo e data di effettivo utilizzo.

7.3.3.8 UploadedDocument

Rappresenta il punto di ingresso dei file fisici nel modulo AI Copilot. Oltre a mantenere i riferimenti di archiviazione (*StoragePath*, *OriginalFilename*, *Checksum*), traccia gli eventuali interventi umani di forzatura dei dati (*OverrideCompany*, *OverrideDepartment*), permettendo di misurare l'affidabilità del sistema automatico.

7.3.3.9 ExtractedDocument

Consolida i dati di business estrapolati dall'intelligenza artificiale a partire dal file originario (*UploadedDocument*). Memorizza le metriche di affidabilità dell'estrazione (*Confidence*), i tempi di elaborazione e mappa il risultato all'entità di business corretta (*MatchedEmployeeId*).

7.3.3.10 ProcessingRun

Entità deputata alla gestione dell'elaborazione asincrona (Background Jobs). Mappa un'esecuzione di sistema (batch o singola) per il processamento dei documenti caricati, esponendo lo stato globale dell'operazione (*Status*, *CompletedAt*, *JobId*) e conteggiando i volumi elaborati (*TotalDocuments*, *ProcessedDocuments*).

7.3.3.11 ProcessingItem

Gestisce la granularità delle operazioni asincrone. Ogni *ProcessingRun* è suddiviso in molteplici *ProcessingItem*, permettendo al sistema di tracciare in dettaglio lo stato di avanzamento (*Status*, *Sequence*) e i potenziali errori (*ErrorMessage*) a livello di singolo documento o operazione isolata.

7.3.3.12 Template

Archivio dei modelli standard precompilati. Contiene strutture di testo riutilizzabili per formattare gli output di sistema o le comunicazioni in uscita in modo omogeneo e standardizzato.

7.3.3.13 Sending

Log di tracciamento per le comunicazioni in uscita. Questa entità collega in modo relazionale un destinatario (*RecipientId*, mappato verso *User*), il contenuto estrapolato dal sistema (*ExtractedDocument*) e l'eventuale formato utilizzato (*Template*), registrando l'effettivo payload della comunicazione (*Subject*, *Body*) e il timestamp di invio (*SentAt*).

8 Requisiti Funzionali Soddisfatti

Il periodo di analisi dei requisiti ha portato all'identificazione di un set di requisiti funzionali che il prodotto software deve soddisfare. Di seguito viene fornita una panoramica dei principali requisiti soddisfatti nel sistema, con riferimento alle sezioni del documento dove sono descritti in dettaglio. I requisiti funzionali si dividono in obbligatori, desiderabili e opzionali, a seconda della loro importanza e priorità per il progetto. In particolare sono stati identificati 132 requisiti funzionali, di cui 97 obbligatori, 0 desiderabili e 35 opzionali. I requisiti obbligatori hanno come focus l'implementazione dei tre moduli principali del sistema, ovvero il modulo di analisi documenti, il modulo di generazione testo e il modulo di raccolta dati, che rappresentano le funzionalità core del prodotto software. Ciascuna di queste aree non era presente all'interno dell'applicativo Nexum quindi i rappresentanti dell'azienda (Eggon) hanno ritenuto fondamentale implementare queste funzionalità per soddisfare le esigenze degli utenti finali e garantire il successo del progetto. I requisiti opzionali, invece, si concentrano su funzionalità come la gestione delle utenze e la distribuzione dei documenti, che invece sono già presenti all'interno dell'applicativo Nexum e quindi non rappresentano un elemento di differenziazione significativo per il prodotto software che stiamo sviluppando.

8.1 Completamento requisiti

- **Requisiti Obbligatori**: 97 requisiti funzionali soddisfatti, con focus sui moduli di analisi documenti, generazione testo e raccolta dati.
- **Requisiti Desiderabili**: 0 requisiti desiderabili identificati.
- **Requisiti Opzionali**: (da definire) requisiti opzionali soddisfatti, principalmente relativi alla gestione utenze e distribuzione documenti.

8.2 Tabella di tracciamento dei requisiti

Tabella 19: Tabella dei Requisiti funzionali$_G$

Codice	Descrizione	Stato	Priorità$_G$
RF-001	Il sistema deve permettere all'utente non autenticato di effettuare la registrazione di un nuovo account.	Implementato	Opzionale$_G$
RF-002	Il sistema deve permettere all'utente di inserire il proprio indirizzo email in fase di registrazione.	Implementato	Opzionale$_G$
RF-003	Il sistema deve permettere all'utente di inserire una password in fase di registrazione.	Implementato	Opzionale$_G$
RF-004	Il sistema deve permettere all'utente di inserire un username in fase di registrazione.	Implementato	Opzionale$_G$
RF-005	Il sistema deve permettere all'utente di inserire il proprio nome in fase di registrazione.	Implementato	Opzionale$_G$
RF-006	Il sistema deve permettere all'utente di inserire il proprio cognome in fase di registrazione.	Implementato	Opzionale$_G$
RF-007	Il sistema deve permettere all'utente di inserire la propria matricola in fase di registrazione.	Implementato	Opzionale$_G$
RF-008	Il sistema deve permettere di visualizzare un messaggio di errore se il formato dell'email inserita non è valido.	Implementato	Opzionale$_G$
RF-009	Il sistema deve permettere di visualizzare un messaggio di errore se la password non rispetta i criteri di sicurezza$_G$.	Implementato	Opzionale$_G$
RF-010	Il sistema deve impedire la registrazione se l'email inserita è già associata a un account esistente.	Implementato	Opzionale$_G$
RF-011	Il sistema deve impedire la registrazione se lo username inserito è già utilizzato.	Implementato	Opzionale$_G$
RF-012	Il sistema deve impedire la registrazione se la matricola inserita è già presente nel sistema.	Implementato	Opzionale$_G$
RF-013	Il sistema deve permettere di visualizzare un messaggio di errore se il formato della matricola non è valido.	Implementato	Opzionale$_G$
RF-014	Il sistema deve permettere all'utente registrato di effettuare il login (autenticazione).	Implementato	Opzionale$_G$
RF-015	Il sistema deve notificare l'errore in caso di tentativo di login con email non registrata o password associata all'email errata.	Implementato	Opzionale$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-016	Il sistema deve consentire all'utente di poter inserire i dati necessari al login.	Implementato	Opzionale$_G$
RF-017	Il sistema deve permettere all'utente di visualizzare i dati del proprio profilo.	Implementato	Opzionale$_G$
RF-018	Il sistema deve mostrare l'email associata al profilo utente.	Implementato	Opzionale$_G$
RF-019	Il sistema deve mostrare (o permettere la gestione del) la password del profilo utente.	Implementato	Opzionale$_G$
RF-020	Il sistema deve mostrare lo username associato al profilo utente.	Implementato	Opzionale$_G$
RF-021	Il sistema deve mostrare il nome associato al profilo utente.	Implementato	Opzionale$_G$
RF-022	Il sistema deve mostrare il cognome associato al profilo utente.	Implementato	Opzionale$_G$
RF-023	Il sistema deve mostrare la matricola associata al profilo utente.	Implementato	Opzionale$_G$
RF-024	Il sistema deve permettere all'utente di effettuare il logout terminando la sessione.	Implementato	Opzionale$_G$
RF-025	Il sistema deve gestire l'uscita dalla modifica profilo senza salvare i cambiamenti.	Implementato	Opzionale$_G$
RF-026	Il sistema deve permettere all'Amministratore$_G$ di visualizzare la lista degli utenti registrati.	Implementato	Opzionale$_G$
RF-027	Il sistema deve permettere la visualizzazione del dettaglio di un singolo utente dalla lista.	Implementato	Opzionale$_G$
RF-028	Il sistema deve mostrare il ruolo associato a un utente registrato.	Implementato	Opzionale$_G$
RF-029	Il sistema deve mostrare il nome di un utente registrato.	Implementato	Opzionale$_G$
RF-030	Il sistema deve mostrare il cognome di un utente registrato.	Implementato	Opzionale$_G$
RF-031	Il sistema deve permettere all'Amministratore$_G$ di modificare il ruolo di un utente registrato.	Implementato	Opzionale$_G$
RF-032	Il sistema deve permettere all'utente di effettuare il logout (terminare la sessione).	Implementato	Opzionale$_G$
RF-033	Il sistema deve generare contenuti testuali tramite AI Assistant$_G$ in base a prompt$_G$ e parametri.	Implementato	Obbligatorio$_G$
RF-034	Il sistema deve permettere l'inserimento di un prompt$_G$ testuale per la generazione.	Implementato	Obbligatorio$_G$
RF-035	Il sistema deve permettere la selezione del tono per la generazione del contenuto.	Implementato	Obbligatorio$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-036	Il sistema deve permettere la selezione dello stile per la generazione del contenuto.	Implementato	Obbligatorio$_G$
RF-037	Il sistema deve permettere la visualizzazione dello storico delle generazioni AI$_G$.	Implementato	Obbligatorio$_G$
RF-038	Il sistema deve notificare l'assenza di elementi se lo storico delle generazioni è vuoto.	Implementato	Obbligatorio$_G$
RF-039	Il sistema deve mostrare i dettagli completi di un elemento selezionato dallo storico.	Implementato	Obbligatorio$_G$
RF-040	Il sistema deve permettere di visualizzare lo stile utilizzato per un contenuto nello storico.	Implementato	Obbligatorio$_G$
RF-041	Il sistema deve permettere di visualizzare il testo del risultato generato nello storico.	Implementato	Obbligatorio$_G$
RF-042	Il sistema deve permettere di visualizzare il timestamp (data/ora) della generazione nello storico.	Implementato	Obbligatorio$_G$
RF-043	Il sistema deve permettere di visualizzare la valutazione assegnata dall'utente al contenuto nello storico.	Implementato	Obbligatorio$_G$
RF-044	Il sistema deve permettere di visualizzare il prompt$_G$ originale utilizzato per un contenuto nello storico.	Implementato	Obbligatorio$_G$
RF-045	Il sistema deve permettere di visualizzare il tono utilizzato per un contenuto nello storico.	Implementato	Obbligatorio$_G$
RF-046	Il sistema deve mostrare un'anteprima del contenuto generato dall'AI$_G$.	Implementato	Obbligatorio$_G$
RF-047	Il sistema deve permettere di modificare l'immagine associata al contenuto generato.	Implementato	Obbligatorio$_G$
RF-048	Il sistema deve notificare l'utente quando tenta di caricare un file immagine non valido.	Implementato	Obbligatorio$_G$
RF-049	Il sistema deve permettere di modificare il titolo del contenuto generato.	Implementato	Obbligatorio$_G$
RF-050	Il sistema deve permettere di modificare il testo del corpo del contenuto generato.	Implementato	Obbligatorio$_G$
RF-051	Il sistema deve permettere di annullare le modifiche apportate al contenuto generato.	Implementato	Obbligatorio$_G$
RF-052	Il sistema deve permettere di riutilizzare i parametri di un contenuto dello storico per una nuova generazione.	Implementato	Obbligatorio$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-053	Il sistema deve permettere di duplicare un contenuto dallo storico per modificarne i parametri.	Implementato	Obbligatorio$_G$
RF-054	Il sistema deve permettere di filtrare la lista delle generazioni nello storico.	Implementato	Obbligatorio$_G$
RF-055	Il sistema deve permettere di visualizzare la lista dello storico aggiornata in base ai filtri applicati.	Implementato	Obbligatorio$_G$
RF-056	Il sistema deve permettere di rigenerare un contenuto tramite AI$_G$ mantenendo i parametri.	Implementato	Obbligatorio$_G$
RF-057	Il sistema deve permettere all'utente di valutare (rating) il contenuto generato.	Implementato	Obbligatorio$_G$
RF-058	Il sistema deve permettere di scartare il contenuto generato e pulire l'interfaccia$_G$.	Implementato	Obbligatorio$_G$
RF-059	Il sistema deve permettere di salvare il contenuto generato nel database.	Implementato	Obbligatorio$_G$
RF-060	Il sistema deve permettere all'utente l'inserimento di un nuovo tono per la generazione di contenuti	Implementato	Obbligatorio$_G$
RF-061	Il sistema deve permettere all'utente l'eliminazione di un tono per la generazione di contenuti	Implementato	Obbligatorio$_G$
RF-062	Il sistema deve permettere all'utente l'inserimento di un nuovo stile per la generazione di contenuti	Implementato	Obbligatorio$_G$
RF-063	Il sistema deve permettere all'utente l'eliminazione di uno stile per la generazione di contenuti	Implementato	Obbligatorio$_G$
RF-064	Il sistema deve permettere l'analisi di documenti tramite il modulo$_G$ AI Co-Pilot$_G$.	Implementato	Obbligatorio$_G$
RF-065	Il sistema deve permettere l'inserimento/selezione della categoria del documento.	Implementato	Obbligatorio$_G$
RF-066	Il sistema deve permettere l'inserimento del mese/anno di competenza del documento.	Implementato	Obbligatorio$_G$
RF-067	Il sistema deve permettere l'inserimento dell'azienda associata al documento.	Implementato	Obbligatorio$_G$
RF-068	Il sistema deve permettere l'inserimento del reparto associato al documento.	Implementato	Obbligatorio$_G$
RF-069	Il sistema deve permettere di controllare la correttezza del formato del file inserito.	Implementato	Obbligatorio$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-070	Il sistema deve permettere di controllare che lo stesso file non sia già stato analizzato.	Implementato	Obbligatorio$_G$
RF-071	Il sistema deve permettere lo split di documenti diversi all'interno dello stesso file.	Implementato	Obbligatorio$_G$
RF-072	Il sistema deve permettere di visualizzare la lista dei documenti analizzati.	Implementato	Obbligatorio$_G$
RF-073	Il sistema deve notificare l'utente se nessun documento è stato riconosciuto dall'analisi.	Implementato	Obbligatorio$_G$
RF-074	Il sistema deve permettere di visualizzare i dettagli di un singolo documento dalla lista.	Implementato	Obbligatorio$_G$
RF-075	Il sistema deve permettere di visualizzare la competenza (periodo) del documento analizzato.	Implementato	Obbligatorio$_G$
RF-076	Il sistema deve permettere di visualizzare l'azienda associata al documento analizzato.	Implementato	Obbligatorio$_G$
RF-077	Il sistema deve permettere di visualizzare la causale del documento analizzato.	Implementato	Obbligatorio$_G$
RF-078	Il sistema deve permettere di visualizzare la lingua rilevata nel documento.	Implementato	Obbligatorio$_G$
RF-079	Il sistema deve permettere di visualizzare il numero di pagine del documento.	Implementato	Obbligatorio$_G$
RF-080	Il sistema deve permettere di visualizzare il nome originale del file del documento.	Implementato	Obbligatorio$_G$
RF-081	Il sistema deve permettere di visualizzare la data di redazione/caricamento del documento.	Implementato	Obbligatorio$_G$
RF-082	Il sistema deve permettere di visualizzare il codice identificativo del documento.	Implementato	Obbligatorio$_G$
RF-083	Il sistema deve permettere di visualizzare la tipologia del documento.	Implementato	Obbligatorio$_G$
RF-084	Il sistema deve mostrare l'anteprima visiva del documento analizzato.	Implementato	Obbligatorio$_G$
RF-085	Il sistema deve permettere di modificare il destinatario associato al documento.	Implementato	Obbligatorio$_G$
RF-086	Il sistema deve permettere di modificare la tipologia del documento.	Implementato	Obbligatorio$_G$
RF-087	Il sistema deve ricalcolare la percentuale di confidenza dopo modifiche manuali.	Implementato	Obbligatorio$_G$
RF-088	Il sistema deve permettere di visualizzare la lista delle informazioni sui destinatari estratti.	Implementato	Obbligatorio$_G$
RF-089	Il sistema deve permettere di visualizzare i dettagli di un singolo destinatario in lista.	Implementato	Obbligatorio$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-090	Il sistema deve permettere di visualizzare il codice fiscale del destinatario.	Implementato	Obbligatorio$_G$
RF-091	Il sistema deve permettere di visualizzare la matricola del destinatario.	Implementato	Obbligatorio$_G$
RF-092	Il sistema deve permettere di visualizzare il reparto del destinatario.	Implementato	Obbligatorio$_G$
RF-093	Il sistema deve permettere di visualizzare il nome/cognome del destinatario.	Implementato	Obbligatorio$_G$
RF-094	Il sistema deve notificare se nessun destinatario è stato riconosciuto.	Implementato	Obbligatorio$_G$
RF-095	Il sistema deve permettere di visualizzare lo storico completo dei documenti processati.	Implementato	Obbligatorio$_G$
RF-096	Il sistema deve notificare l'assenza di documenti nello storico.	Implementato	Obbligatorio$_G$
RF-097	Il sistema deve permettere di visualizzare i dettagli di un elemento nello storico documenti.	Implementato	Obbligatorio$_G$
RF-098	Il sistema deve permettere di visualizzare la percentuale di confidenza dell'analisi nello storico.	Implementato	Obbligatorio$_G$
RF-099	Il sistema deve permettere di visualizzare l'appartenenza alle liste di distribuzione.	Implementato	Obbligatorio$_G$
RF-100	Il sistema deve permettere di visualizzare lo stato di elaborazione del documento.	Implementato	Obbligatorio$_G$
RF-101	Il sistema deve permettere di caricare un template$_G$ di messaggio esistente.	Implementato	Obbligatorio$_G$
RF-102	Il sistema deve permettere di modificare l'oggetto del messaggio.	Implementato	Obbligatorio$_G$
RF-103	Il sistema deve permettere di modificare il testo del corpo del messaggio.	Implementato	Obbligatorio$_G$
RF-104	Il sistema deve permettere di salvare il messaggio corrente come nuovo template$_G$.	Implementato	Obbligatorio$_G$
RF-105	Il sistema deve permettere di eliminare un template$_G$ di messaggio.	Implementato	Obbligatorio$_G$
RF-106	Il sistema deve permettere di visualizzare la lista dei template$_G$ di messaggio disponibili.	Implementato	Obbligatorio$_G$
RF-107	Il sistema deve permettere di visualizzare un elemento della lista dei template$_G$ di messaggio disponibili.	Implementato	Obbligatorio$_G$
RF-108	Il sistema deve permettere di visualizzare l'oggetto del template$_G$.	Implementato	Obbligatorio$_G$
RF-109	Il sistema deve permettere di visualizzare il testo del template$_G$.	Implementato	Obbligatorio$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-110	Il sistema deve permettere di visualizzare il codice del template$_G$.	Implementato	Obbligatorio$_G$
RF-111	Il sistema deve permettere l'invio del documento e del messaggio associato.	Implementato	Opzionale$_G$
RF-112	Il sistema deve permettere di allegare ulteriore contenuto al messaggio.	Implementato	Opzionale$_G$
RF-113	Il sistema deve permettere di pianificare l'invio del documento e del messaggio associato.	Implementato	Opzionale$_G$
RF-114	Il sistema deve permettere il filtraggio della lista dei documenti analizzati.	Implementato	Obbligatorio$_G$
RF-115	Il sistema deve mostrare la lista dei documenti aggiornata in base ai filtri.	Implementato	Obbligatorio$_G$
RF-116	Il sistema deve permettere il filtraggio della lista dei destinatari.	Implementato	Obbligatorio$_G$
RF-117	Il sistema deve mostrare la lista dei destinatari aggiornata in base ai filtri.	Implementato	Obbligatorio$_G$
RF-118	Il sistema deve permettere il filtraggio della lista dello storico documenti.	Implementato	Obbligatorio$_G$
RF-119	Il sistema deve mostrare la lista dello storico documenti aggiornata in base ai filtri.	Implementato	Obbligatorio$_G$
RF-120	Il sistema deve permettere di mostrare l'audit di un documento nello storico.	Implementato	Obbligatorio$_G$
RF-121	Il sistema deve permettere di visualizzare la dashboard$_G$ con i dati di analytics per l'AI Assistant$_G$.	Implementato	Obbligatorio$_G$
RF-122	Il sistema deve permettere di visualizzare il numero totale di prompt$_G$ generati.	Implementato	Obbligatorio$_G$
RF-123	Il sistema deve permettere di visualizzare il rating medio dei prompt$_G$ generati.	Implementato	Obbligatorio$_G$
RF-124	Il sistema deve permettere di visualizzare il numero di rigenerazioni effettuate.	Implementato	Obbligatorio$_G$
RF-125	Il sistema deve permettere di visualizzare le statistiche sui toni più utilizzati.	Implementato	Obbligatorio$_G$
RF-126	Il sistema deve permettere di visualizzare le statistiche sugli stili più utilizzati.	Implementato	Obbligatorio$_G$
RF-127	Il sistema deve permettere di visualizzare la dashboard$_G$ con i dati di analytics per l'AI Co-Pilot$_G$.	Implementato	Obbligatorio$_G$
RF-128	Il sistema deve permettere di visualizzare la confidenza media delle analisi documenti.	Implementato	Obbligatorio$_G$
RF-129	Il sistema deve permettere di visualizzare la percentuale di interventi manuali necessari.	Implementato	Obbligatorio$_G$

Continua nella prossima pagina...

Tabella 19 – continua dalla pagina precedente

Codice	Descrizione	Stato	Priorità$_G$
RF-130	Il sistema deve permettere di visualizzare l'accuratezza del mapping dei dati.	Implementato	Obbligatorio$_G$
RF-131	Il sistema deve permettere di visualizzare i tempi medi di analisi dei documenti.	Implementato	Obbligatorio$_G$
RF-132	Il sistema deve permettere di filtrare i dati di analytics per periodo temporale.	Implementato	Obbligatorio$_G$